

F18A



ASYNCHRONOUS
144-NODE
ARCHITECTURE

ASSEMBLY PROGRAMMING GUIDE

Low-Level Programming for the Asynchronous Future



; F18A ASSEMBLY EXAMPLE

```
0000 START: LDI R1, #00FFh
0002         LDI R2, #0100h
0004 LOOP:  ADD R1, R2
0006         ST  [R3], R1
0008         INC R3
000A        DJNZ R2, LOOP
000C        JMP  HALT
000E HALT:  WFI
0010        JMP  HALT
```

; --- END ---

F18A REGISTER SET

R0 General Purpose
R1 General Purpose
R2 General Purpose
R3 Pointer / Index
R4 General Purpose
R5 General Purpose
R6 Status / Flags
R7 Return Address
PC Program Counter
SP Stack Pointer
FL Flags Register

FLAGS (FL)

N	Z	C	V	I	M	R
---	---	---	---	---	---	---



DETAILED
INSTRUCTION SET



PROGRAMMING
EXAMPLES



ASYNCHRONOUS
ARCHITECTURE



EFFICIENT
BY DESIGN

1. Introduction

The F18 technology consists of small, simple computers and several types of versatile, software defined I/O. The GA144 chip is configured as an array of F18 computers with the appropriate selections of I/O types.

An F18 computer is a physically small, very fast, energy-efficient and inexpensive 18-bit machine. Each F18 is self-contained as a complete system, with its own RAM, ROM, stacks, registers, and where appropriate external I/O. The F18 computers in an array operate independently and asynchronously, communicating with each other using efficient internal communication ports.

This book will familiarize you with the architecture, internal structure, and use of the computers and I/O of chips built from F18A technology. Please refer to the data books for specific chips using this technology to learn their exact configurations, pin-outs, ROM content and electrical characteristics. The current editions of these, along with many other relevant documents and application notes as well as the current edition of this document, may be found on our website. It is always advisable to ensure that you are using the latest documents before starting work.

1.1 Status of Data Given

The data given herein are PRODUCTION. Electrical and timing values are illustrative only. Otherwise, the subject matter of this book is the architecture, instruction set, and programming of the F18A computers and I/O, which are fixed by design. Future revisions of this book may document upward-compatible enhancements of the F18A programming model.

1.2 Documentation Conventions

1.2.1 Numbers

Numbers are written in decimal unless otherwise indicated. Hexadecimal values are indicated by explicitly writing "hex" or by preceding the number with the lowercase letter "x".

1.2.2 Node coordinates

Each GreenArrays chip is a rectangular array of **nodes**, each of which is an F18 computer. By convention these arrays are visualized as seen from the top of the silicon die, which is normally the top of the chip package, oriented such that pin 1 is in the upper left corner. Within the array, each node is identified by a three or four digit number denoting its Cartesian coordinates within the array as *yx*x or *yyxx* with the lower left corner node always being designated as node 000. Thus, for a GA144 chip whose computers are configured in an array of 18 columns and 8 rows, the numbers of the nodes in the lower right, upper left, and upper right corners are 017, 700, and 717 respectively.

1.2.3 Register names

Register names in prose may be used with or without the word "register" and are usually shown in a bold font and capitalized where necessary to avoid ambiguity, such as for example the registers **T S R I A B** and **IO** or **io**.

1.2.4 Bit Numbering

Binary numbers are visualized as a horizontal row of bits, numbered consecutively right to left in ascending significance with the least significant bit numbered zero. Thus bit *n* has the binary value 2^n . The notation P9 means bit 9 of register **P**, whose binary value is x200, and T17 means the sign (high order) bit of 18-bit register **T**.

2. The F18A Computer

Here is an introduction to the resources within each F18A computer. All elements are 18 bits wide unless otherwise noted.

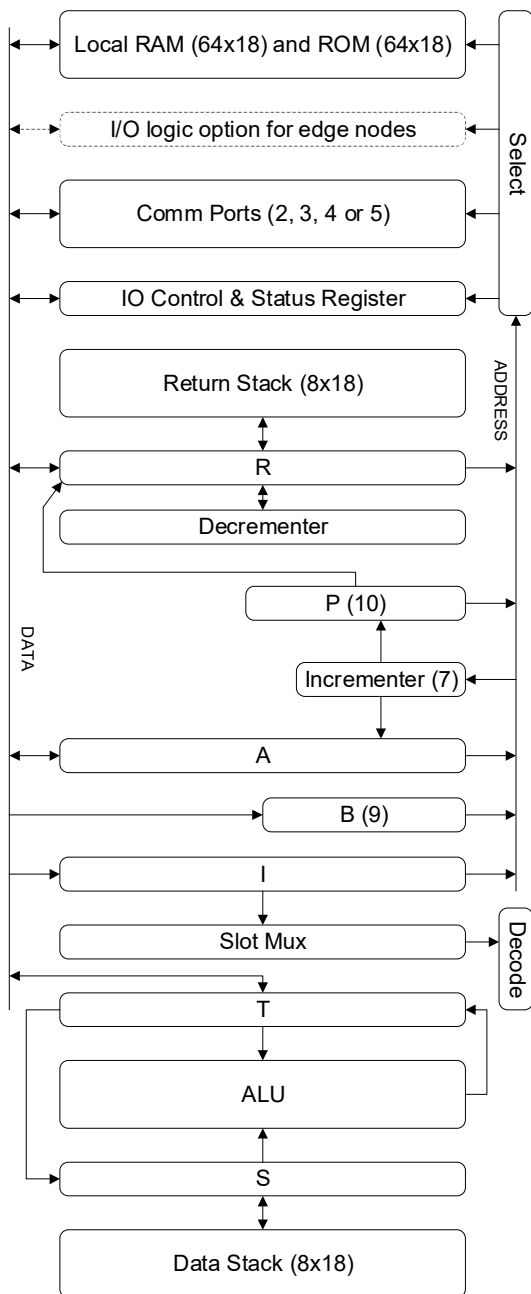


Figure 1 F18A Block Diagram

- **RAM:** 18 bits wide, 64 words in each node.
- **ROM:** 18 bits wide, 64 words in each node.
- **I/O logic** may be present on any edge node of a chip.
- **Communication Ports** are configured based on position within the array (four for inner; three ports array edges; two for corners.) One additional port is optional.
- **IO** control and status for communication ports and I/O logic.
- **Return Stack** consists of 8 registers addressed circularly (no fixed top or bottom.) Pushing a word onto this stack makes it the new “top” item, overwriting the former “bottom” item. After popping 8 items, those 8 items are repeated.
- **R** extends this stack to 9 entries but only 8 are circular. When used as a loop counter, **R** decrements toward zero.
- **P** serves as a “program counter”, holding the address of the next word in the instruction stream. Within RAM or ROM, it is incremented after each word is fetched, circularly within whichever storage class it currently points to. When **P** addresses I/O space, it is not incremented. **Bit P9** enables Extended Arithmetic Mode when set.
- **A** is a general purpose read/write address or data register.
- **B** is a write only address register, containing on reset the address of **io** register.
- **I** is the instruction register into which instruction words containing 1, 2, 3 or 4 opcodes are fetched for execution.
- **T** is the top word of a 10-element data stack.
- **The ALU** (Arithmetic/Logic unit) performs unary operations on **T**, binary operations on **S** and **T**, or a multiply step involving **S**, **T**, and **A** as directed by opcodes. A carry latch is involved in some Extended Arithmetic mode instructions.
- **S** is the second word of a 10-element data stack.
- **Data Stack** is 8 circularly addressed registers constituting the third through 10th elements of a 10-element stack.

2.1 After Reset

On chip reset, the following registers are initialized:

P is set to the address configured in the chip layout. For F18A, this will always be either a multiport execute or will be ROM address **x0aa**.

io is set to the state it would have after a program wrote **x15555** into the register.

B is set to the address of **io**.

Stack pointers are set to the same initial condition on every reset.

Other registers, stack contents, and RAM are not directly affected by reset.

2.2 Memory Addressing

The F18A decodes a nine-bit memory address as shown in Figure 2.

Start	End	Address Space
x000	x03F	RAM, 64 words
x040	x07F	RAM x000-x03F repeated
x080	x0BF	ROM, 64 words
x0C0	x0FF	ROM x080-x0BF repeated
x100	x1FF	I/O ports and registers

Figure 2 Address Map

Address incrementing, when applied to A or P, continues from x03F to x040 but wraps from x07F to x000. The same process occurs with addresses in the ROM space. Incrementing does not occur at all when the address lies in the region of I/O ports and registers. Incrementing never affects bits P8 or P9. P9 has no effect on memory decoding; it simply enables the Extended Arithmetic Mode when set.

2.3 Instruction Set

The F18 instruction set consists of 5-bit opcodes as summarized in Figure 3:

Opcode	Notes	Opcode	Notes
;	00 return	+	10 multiply step
ex	01 execute (swap P and R)	2*	11 left shift
name ;	02 jump to <i>name</i>	2/	12 right shift (signed)
name	03 call to <i>name</i>	inv	13 invert all bits (<i>was -</i>)
unext	04 loop within I (decrement R)	+	14 add (or add with carry)
next	05 loop to address (decrement R)	and	15
if	06 jump if T=0	xor	16 exclusive or (<i>was or</i>)
-if	07 jump if T≥0	drop	17
@p	08 literal (autoincrement P)	dup	18
@+	09 fetch via A (autoincrement A)	r>	19 pop R push T (<i>was pop</i>)
@b	0A fetch via B	over	1A
@	0B fetch via A	a	1B fetch <i>from</i> register A
!p	0C store via P (autoincrement P)	.	1C nop
!+	0D store via A (autoincrement A)	>r	1D pop T push R (<i>was push</i>)
!b	0E store via B	b!	1E store <i>into</i> register B
!	0F store via A	a!	1F store <i>into</i> register A

The shaded opcodes may be used in slot 3 of an instruction word (see 2.3.1 below). Add (+) and multiply step (+*) behave differently in Extended Arithmetic Mode.

2.3.1 Instruction Word Format

Each F18A instruction word is subdivided into four possible slots for opcodes that are normally executed sequentially left to right when viewing the word as conventionally oriented with most significant bit to the left. Jump instructions may be written in slots 0, 1, or 2, and when they are encountered the remainder of the instruction word is interpreted as an address field. This results in four possible instruction word formats as shown in Figure 4:

Slot 0					Slot 1					Slot 2					Slot 3		
Instruction 0					Instruction 1					Instruction 2					Instruction 3		
Instruction 0					Instruction 1					Jump instruction					Destination		
Instruction 0					Jump instruction					Destination							
Jump instruction					Unused			Destination									
17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Figure 4 Instruction word formats

The n-bit destination address field simply replaces the n low order bits of the current (incremented) value of P at the time the jump instruction executes. Thus the scope of a slot 2 jump is very limited while a slot 0 jump can reach any addressable destination, and can control P9 to explicitly enable or disable Extended Arithmetic Mode. Slot 1 and 2 jumps have no effect upon P9, so its selection is unchanged as these jumps are executed. In the F18A, slot 1 and 2 jumps additionally force P8 to zero. This means the destination of any slot 1 or 2 jump may never be in I/O space.

The instruction selected for slot 3 must be one of those shaded in Figure 3 above. When the desired opcode cannot be fit into the current instruction word, either because it may not be used in slot 3 or because it is a jump whose destination requires a longer address field, the remaining slots of the current word may be filled with **nop** instructions.

In addition to jumps, three other instructions affect the consecutive execution of opcodes within an instruction word:

; (return) and **ex** (execute) discontinue execution of the current instruction word.

unext (micronext) conditionally causes execution to loop back to slot 0 (and continue with consecutive slots). When the loop is complete, execution continues with the slot following **unext**.

The arrayForth compiler formats instruction words correctly and automatically. If you wish to compile your own instruction words, to modify code in memory, or to write your own compiler for the F18A, please refer to section 2.4.4 for details about instruction encoding.

2.3.2 Stack Operation

In all cases of pushing and popping stacks mentioned above, the mechanism is the same:

Pushing data stack: S is pushed onto the circular stack, replacing its bottom entry, and T is copied into S.

Popping data stack: S is copied into T unless the instruction states that it sets T to some other value, and the top entry of the circular stack is copied into S.

Pushing return stack: R is pushed onto the circular stack, replacing its bottom entry.

Popping return stack: The top entry of the circular stack is copied into R.

Pushing and popping of the circular stacks is accomplished by adjusting pointers. When popping a circular stack, the previous top item becomes the new bottom item. When pushing onto a circular stack, the new top value replaces the previous bottom value.

2.3.3 Arithmetic, Logic and Register Manipulation

There are sixteen instructions in this class. Approximate execution times (see section 2.4.1) are 1.5 nanoseconds.

Op	Source	Function
x10	+*	Multiply step. Signed Multiplicand is in S, unsigned multiplier in A, T=0 at start of a step sequence. Uses T:A as a 36-bit shift register with multiplier in A. Does the following: 1. If bit A0 is zero, shifts the 36-bit register T:A right one bit arithmetically (T17 is not changed and is copied into T16. T0 is copied to A17 and A0 is discarded.) 2. If bit A0 is one, S and T are added as though they had both been extended to be 19 bit signed numbers, and the 37-bit concatenation of this sum and A is shifted right one bit to replace T:A. <i>Overflow may occur if S and T are both nonzero and their signs differ; this can only occur through improper initialization of T.</i> This instruction is affected in Extended Arithmetic Mode by including the latched carry in the sum in case (2) above, and by latching the carry out from bit 17 of the sum, but this is not a particularly useful effect and may be changed in later F18 versions. <i>You must arrange that the previously executed instruction has not changed the values of S, T or P9. Use nop preceding Multiply Step if necessary to meet this condition.</i>
x11	2*	Two-Star. Shifts T left one bit logically (shifts zero into T0, discards T17) thus multiplying a signed or unsigned value by two.
x12	2/	Two-Slash. Shifts T right one bit arithmetically (propagates T17 by leaving it unchanged; discards T0) thus dividing a signed value by two and discarding the positive remainder.
x13	inv	Invert. Inverts each bit of T, replacing T with its ones complement.
x14	+	Plus. Replaces T with the twos complement sum of S and T. Pops data stack into S. This instruction is affected in Extended Arithmetic Mode, becoming Add with carry . Includes the latched carry in the sum, and latches the carry out from bit 17. <i>You must arrange that the previously executed instruction has not changed the values of S, T or P9. Use nop preceding Plus if necessary to meet this condition. However, if + is used in slot 3 of an instruction word, the (pre-)fetching of the next instruction word increases execution time of the slot 3 add so that it need not be preceded by nop.</i>
x15	and	Replaces T with the Boolean AND of S and T. Pops data stack.
x16	xor	Exclusive Or. Replaces T with the Boolean XOR of S and T. Pops data stack.
x17	drop	Drops the top item from the data stack by copying S into T and popping the data stack.
x18	dup	Duplicates the top item on the data stack by pushing the data stack and copying T into S.
x19	r>	Moves R into T, popping the return stack and pushing the data stack.
x1A	over	Makes a copy of S on top of the data stack by pushing S onto the stack, moving T into S, and replacing T by the previous value of S.
x1B	a	Fetches the contents of register A into T, pushing the data stack.
x1C	.	Nop. Spends time while making no explicit state changes in registers or stacks.
x1D	>r	Moves T into R, pushing the return stack and popping the data stack.
x1E	b!	B-Store. Stores T into register B, popping the data stack.
x1F	a!	A-Store. Stores T into register A, popping the data stack.

Depending on their arguments, any of these may alter T and/or S, with the sole exception of **nop**.

The carry latch is not affected by reset and its state on power-up is unpredictable.

2.3.4 Memory Read and Write

There are eight instructions in this class. Approximate execution times (see section 2.4.1) are 5.1 nanoseconds when reading or writing internal memory. The addresses used may be in memory or I/O spaces; some I/O addresses may suspend execution until the conditions necessary for their completion are met. The notation [X] should be read in the descriptions below as “the memory or I/O addressed by X”.

Op	Source	Function
x08	@p	Fetch-P. Pushes data stack, reads [P] into T, and increments P (see 2.2)
x09	@+	Fetch-plus. Pushes data stack, reads [A] into T, and increments A (see 2.2)
x0A	@b	Fetch-B. Pushes data stack and reads [B] into T.
x0B	@	Fetch. Pushes data stack and reads [A] into T.
x0C	!p	Store-P. Writes T into [P], pops the data stack, and increments P (see 2.2)
x0D	!+	Store-plus. Writes T into [A], pops the data stack, and increments A (see 2.2)
x0E	!b	Store-B. Writes T into [B] and pops the data stack.
x0F	!	Store. Writes T into [A] and pops the data stack.

Figure 6 Memory Read and Write Instructions

2.3.5 Transfer of Program Control

There are eight instructions in this class. Approximate execution times (see section 2.4.1) are 5.1 nanoseconds when the destination is in internal memory. **unext** takes approximately 2.0 nanoseconds. When P is changed by one of these instructions, the new destination may be in memory or I/O space. Some I/O addresses may suspend execution until the conditions necessary for their completion are met.

Op	Source	Function
x00	;	Return. Moves R into P, popping the return stack. Skips any remaining slots and fetches next instruction word.
x01	ex	Execute. Exchanges R and P, skips any remaining slots and fetches next instruction word.
x02	name ;	Jump. Sets P to destination address (see 2.3.1) and fetches next instruction word.
x03	name	Call. Moves P into R, pushing an item onto the return stack, sets P to destination address (see 2.3.1) and fetches next instruction word.
x04	unext	Micronext. If R is zero, pops the return stack and continues with the next opcode. If R is nonzero, decrements R by 1 and causes execution to continue with slot 0 of the current instruction word without re-fetching the word.
x05	next	Next. If R is zero, pops the return stack and continues with the next instruction word addressed by P. If R is nonzero, decrements R by 1 and jumps (see 2.3.1).
x06	if	If. If T is nonzero, continues with the next instruction word addressed by P. If T is zero, jumps (see 2.3.1).
x07	-if	Minus-if. If T is negative (T17 set), continues with the next instruction word addressed by P. If T is positive, jumps (see 2.3.1).

Figure 7 Control Transfer Instructions

2.4 Fine Points

For full mastery of the F18A computer, there are several additional things you should understand.

2.4.1 Execution Time

The F18A is built from asynchronous logic, therefore its instruction times are naturally approximate. The time required for all activity varies directly with temperature, inversely with power supply voltage (V_{DD}), and randomly within a statistical distribution due to variations in the process of fabricating the chips themselves. Additional variations may occur with minor revisions in the version of F18 technology used by a particular chip. Applications should not be designed to depend upon the speed of activities within F18 computers for timing purposes without taking all of these variables into consideration.

The time required for interaction with entities outside an F18 computer (other nodes, or I/O) naturally depends on the behavior of those outside devices.

The approximate execution times shown in the text above are typical of the F18A. For any particular chip, see its Product Briefs and Data Books for pertinent preliminary information from prototype chips and, later, statistical data from measurements on production chips.

2.4.2 Prefetch considerations

Any F18A computer may, when it is logically possible to do so, be designed to fetch instructions before they are needed. We reserve the right to make changes or improvements in this process with each chip we design, and therefore you are advised, in the interest of software portability, *not* to depend on any particular prefetching strategy for the timing of your program or for avoiding the need to insert a **nop** between an instruction that alters S, T or P9 and an **add** or **multiply step** instruction. If your requirements justify sacrificing upward compatibility in favor of taking such steps to achieve maximum performance, contact our support staff (hotline mailbox) for recommendations pertinent to specific chips.

2.4.3 Best Practices

Within I/O space, read and write only those addresses defined to be readable or writable to avoid unexpected results.

Reading or writing inappropriate I/O addresses, or writing ROM, may result in a hung computer.

Upward compatibility to future versions of the F18 is not guaranteed. Think in terms of source, not binary compatibility. Be aware of the reset state of the **io** register since that will be the default setting for any bits assigned to control future I/O facilities.

2.4.4 Compiling Instructions Yourself

If you are writing a compiler, or wish to compose F18A instructions by hand or to read or modify instruction words in memory, you must be aware that instruction words are *encoded* as follows:

Compose instruction words as shown in 2.3.1 with opcode values as shown in the “Op” columns of the instruction tables above and exclusive OR the result with the value x15555. If the word requires a destination address field, mask that field to zero and insert an un-encoded address value consistent with those shown in 2.2 and the specific I/O addresses defined in section 3. In the particular case of slot zero jumps, it is permissible to set unused bits 10, 11 and 12 to any convenient states.

3. F18A I/O Facilities

Input-output capabilities of F18A nodes consist of internode communication ports (2, 3, 4, or 5 depending on array configuration and the node's position in the array) and, for nodes on array edges, optional GPIO pins and/or other more specialized circuitry such as parallel buses, analog I/O, or serializer-deserializer (SERDES).

Name	Addr	Description	Notes
io	x15D	I/O control and status (see below). Bit assignments depend on the facilities present.	
data	x141	Up port with no handshake.	3
---u	x145	Up	1
ldata	x171	Left port with no handshake.	3
--l-	x175	Left	2
--lu	x165	Left or Up	1,2
-d--	x115	Down	
-d-u	x105	Down or Up	1
-dl-	x135	Down or Left	2
-dlu	x125	Down, Left or Up	1,2
r---	x1D5	Right	
r--u	x1C5	Right or Up	1
r-l-	x1F5	Right or Left	2
r-lu	x1E5	Right, Left or Up	1,2
rd--	x195	Right or Down	
rd-u	x185	Right, Down or Up	1
rdl-	x1B5	Right, Down or Left	2
rdlu	x1A5	Right, Down, Left or Up	1,2

Figure 8. I/O Address Table

Figure 8 shows all of the defined I/O addresses in the F18A. Shaded addresses are not *present* in all nodes; see the numbered notes for more information.

Addresses should only be used in a given node if they are *present* according to these notes; right, down, left and up ports are present for read and write in all nodes except as noted:

Blank: *Present* for read/write in all nodes.

- 1 *Absent* on top and bottom nodes unless supported by particular I/O circuitry
- 2 *Absent* on side nodes unless supported by particular I/O circuitry
- 3 *Present* only when documented as such for specific I/O circuitry; read, write, and handshaking facilities are defined by each class of circuitry.

3.1 The io Register

This 18-bit control and status register is each F18A computer's interface with its I/O circuitry, if any, and with handshake lines for any communication ports that are present. The bit assignments are shown for reference here. Their semantics are defined in detail in later sections. By default, **io** is populated with latches that read as the inverse of what was last written to them. Any **io** bits that are undefined for a given node because the associated I/O facilities are absent implement this behavior. A second general principle is that on reset each node's **io** register is set as though the value x1555 had been written to it under program control.

Bit	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
reset	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
WRITE	GPIO pin 17 control						WD		phan 9		phan 7		GPIO pin 5 control		GPIO pin 3 control		GPIO pin 1 control	
Alt write	SR			A/D mode		DB			9-bit D/A value XORed with x155									
READ	pin 17	Rr-	Rw	Dr-	Dw	Lr-	Lw	Ur-	Uw				pin 5		pin 3		pin 1	

Figure 9. IO Register Bit Assignments

3.2 Suspension

A node is either executing instructions at full speed or it is *suspended* in mid-instruction, waiting for a “wake up” condition that depends on the state of another node or of other I/O circuitry. Since an F18 has no clock, the acts of suspending and waking up consume no energy or time in and of themselves. Perception of changing conditions takes picoseconds within the chip and, typically, nanoseconds for events external to the chip. When a node is suspended the power used by its computer is nil, on the order of 100 nanowatts. When executing, an F18 computer uses power on the order of less than 6 milliwatts depending on the instructions used and the data on which they are operating.

3.3 Communication Ports

Communication (comm) ports provide for direct, point to point bidirectional information transfer within a chip. Each F18A node may have from two to five ports connected. Comm ports are named **Right, Down, Left, Up**; the fifth port is not implemented in F18A based chips.

Each comm port is an independent, bidirectional, half duplex 18-bit channel connecting two nodes. Communication occurs when a sending node writes to a comm port address and a receiving node to which that port is connected reads the corresponding comm port address at the same time. When a node operates on a port the data transfer occurs at approximately memory speed unless the other node connected to the port is not yet performing the complementary operation; in this case the operating node suspends, waking up when the other node connected to that port is performing the complementary operation.

The status of each port is visible through the bits in **io** labeled **R/D/L/Ur-** and **R/D/L/Uw** above. **Dr-** is zero if the node at the other end of **Down** is reading; **Lw** is one if the node at the other end of **Left** is writing.

3.3.1 Multiport Operations

As shown in Figure 8 above, the I/O addressing scheme of the F18A permits comm ports to be combined. Reading or writing **Multiport Addresses** such as **rd-u** is permitted and suspends until one or more of the nodes to which the selected ports connect is performing the complementary operation. The design rules for communication flow using multiport operations are straightforward: When a node is doing a multiport read, only one of the selected ports may be written by another node at any given time. When a node is doing a multiport write, every node that intends to read the value written must already be doing so and suspended. It is not possible to directly determine which port(s) transferred data after a multiport operation by inspection of **io**; if it is necessary for a node to make this determination it must be accomplished via protocols.

3.3.2 Port Execution

An F18 may execute instruction streams directly from a comm port. When the F18 fetches an instruction word from a comm port, **P** is not incremented and unless the instruction stream transfers control elsewhere subsequent words are fetched from the same port. **@p** reads a word from the port and **!p** writes a word into the port. Multiport addresses may be used so long as the design rules in the preceding section are followed.

3.4 Nodes with GPIO Pins

Any edge node may have up to four fully featured General Purpose I/O (GPIO) pins. A schematic is shown in Figure 10. These pins have useful electrical characteristics and are designed to facilitate software defined I/O. GPIO pins are designated by their bit positions as shown in Figure 9 above (pins 17, 5, 3 and 1.)

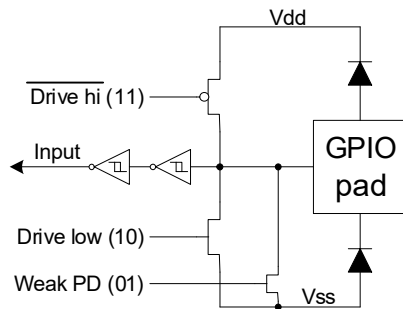


Figure 10 F18A GPIO Pin Circuit

Under program control, you may place each pin in one of four states by writing a value into its 2-bit GPIO field of **io** as follows:

Value	State of pin drive circuitry
00	High Impedance (≈ 2.8 pF, ≈ 250 M Ω)
01	Weak pulldown (≈ 38 μ A in saturation). This is the reset state of all GPIO pins.
10	Drive to Vss (≈ 41 mA in saturation)
11	Drive to Vdd (≈ 41 mA in saturation)

Regardless of the drive state selected for a GPIO pin, you may always read the **io** register to measure the actual voltage on the pin as a logic state (1=pin voltage high) using bits 17, 5, 3 or 1.

Input hysteresis, provided by two Schmitt triggers, protects against noise and other classical problems with signals whose rise and fall times are long relative to computer speeds. The F18A computers and I/O circuitry are fast and sensitive enough that parasitic resistance, inductance and capacitance effects may be directly observed by reading the pins as external circuits are being driven, and are fast enough that a 1 MHz sine wave is a slow-moving signal. Refer to the documentation on specific chips for electrical characteristics of their GPIO pins.

3.4.1 Wakeup Pins

When an edge node is configured with a GPIO pin 17 it may be configured and documented for wakeup on that pin. When this is the case, the pin may be “read” using a comm port address which becomes *present for reading*: **Up** for nodes on the top and bottom edges of the array, or **Left** for nodes on the lateral sides. The node suspends until the pin is in the state specified by the **WD** bit in the **io** register. When **WD** is written as zero, which is the reset state of that bit, a pin read suspends until the pin is logically high. To suspend until the pin is low, write 1 to **WD**. *When the read completes, it is required that you must immediately **drop** the value that was read into T.*

F18A multiport read operations may include the wakeup pin as **Up** or **Left**. *In this case you must still immediately **drop** the value read.* To deduce that the pin was the source of the wakeup it is necessary to read **io** and check the current state of bit 17.

3.4.2 Shared (Read-only) Pins

Some nodes may be configured to *observe* the voltage on a GPIO pin that they do not directly control. This is typically done to provide versatility in timing analog to digital conversion. In these cases, the shared pin may be read as bit 17 of **io**. Wakeup may be enabled for reading, writing, or both on a shared pin, as will be documented for the type of I/O node observing the pin. Wakeup polarity is controlled by **WD** as with GPIO pins. [See the chip Data Book for details on each node's wakeup configuration as the choice of this configuration may depend on their intended uses].

3.4.3 Phantom Wakeup Signals

A *sending* node may be configured to drive a “phantom” signal that a *receiving* node treats as a shared pin. In this configuration the receiving node operates exactly as described above in 3.4.2. The sending node controls the phantom signal by writing into bits **phan 7** or **phan 9**; writing 1 makes the signal look high to the receiver, 0 makes it look low, and the reset state of **io** is equivalent to writing zeroes (low) into these bits.

3.5 Parallel Bus Support

F18A Parallel Bus support consists of a cluster of three adjacent nodes, typically laid out on the “bottom” edge of a chip. The center node has 4 GPIO pins for control signals and each of its parallel port neighbors has I/O circuitry for an 18-bit bidirectional bus interface. The purposes of all these pins and the direction and timing of signals are defined by software.

The center node is configured to act as sender of a phantom wakeup signal to each of the parallel bus nodes to supplement the comm ports between them to provide versatility in timing.

In each node with an 18-bit bus, the addresses **Up** and **Data** are enabled for reading and writing. The difference between these two addresses is that operations on **Up** suspend when appropriate (see 3.4.3) while operations on **Data** do not depend upon the phantom wakeup signal.

Each of the 18 pins of a parallel port node has a subset of the capabilities of a GPIO pin. When **DB** in **io** is set to 1 (its reset state) all 18 pins are in output mode and are driven high or low depending on the state of an 18-bit Bus Register. When **DB** is set to zero, all 18 pins are set to high impedance input mode. Unlike GPIO pins, parallel bus pins *do not* have Schmitt triggers, nor do they have weak pull-down capabilities.

Writing to **Up** or **Data** always sets the Bus Register, regardless of the state of **DB**. Writing a 1 or 0 to a bit in this register will, in output mode, drive the corresponding pin to VDD or Vss, respectively. Reading from **Up** or **Data** in input mode reads the pin states with high reading as 1; in output mode, reading returns the current value of the Bus Register.

Note that parallel buses reset to output mode and that reset does not initialize the Bus Register. Its value after power up is unpredictable.

Refer to the documentation on specific chips for electrical characteristics of their parallel bus pins.

3.6 SERDES Nodes

The high speed Serializer-Deserializer (SERDES) node is equipped with a transceiver for 2-wire (clock and data) synchronous, half duplex communications at a rate of approximately 450 megabits per second. While a SERDES node might be capable of communicating with something other than another SERDES node, it was not designed with any other objective than communication between our chips.

To receive an 18-bit word, a SERDES node must place the value x3FFFE in T and read the **Up** port. The node suspends until a word has been received.

To begin transmitting, a SERDES node must write the first word to **Data** and must then set bit 17 of **io** to 1. Each subsequent word is written to **Up**; the node will suspend until the preceding word has been finished and transmission of the new word may begin. If the node wishes to stop transmitting cleanly, it must wait long enough after writing the last word to **Up** (or after setting **io** bit 17 if only one word is to be transmitted) for a full word to have been sent.

Refer to the documentation on specific chips for electrical and timing characteristics of their SERDES nodes and pins.

3.7 Analog Nodes

Any F18A edge node may be configured with an analog input pin and an analog output pin along with the associated circuitry. A block diagram is shown in Figure 11.

The Analog to Digital converter (ADC) consists of a high speed, free running down-counter whose frequency varies as a function of input voltage between ≈ 3.6 GHz for V_{DD} input and ≈ 5.6 GHz for V_{SS} , as shown in the typical curve in Figure 13. The **A/D Mode** field in **io** controls the counter as follows:

Value	Mode and input of ADC counter
00	High Impedance ADC input pad (≈ 2.8 pF, ≈ 250 M Ω)
01	V_{DD} Calibration
10	Counter disabled to save power (the reset state)
11	V_{SS} Calibration

A voltage is measured within the operating range (≈ 750 mV to ≈ 1.3 v) by calculating the difference between two readings separated by a known time interval.

Figure 11 Analog I/O Circuitry

For analog nodes on top and bottom edges, **Data** and **Up** are present for reading and writing; for those on sides of an array, **Ldata** and **Left** are present. The procedure for sampling the ADC requires two steps: First *write* to the **Up/Left** or **Data/Ldata** port as appropriate to signal the VCO to stop for reading, and then *read* from **Data/Ldata** or **Up/Left** as appropriate to obtain the value of the down-counter and restart the VCO. Reading without writing first will not yield useful values. An analog node is equipped with a shared wakeup pin (see 3.4.2) and configured to suspend when writing to **Up/Left**.

The Digital to Analog Converter (DAC) is a programmable current source designed to produce a voltage across a resistance to ground or to source an op-amp. By writing the xor of a desired *value* and x155 into the low order nine bits of **io** that *value* controls the DAC output. A value of x1FF (written as x0AA) sets the DAC for maximum current; a value of 0 (written as x155, the reset state) sets minimum current, high impedance output. Typical DAC transfer functions, in mV versus DAC *values*, into 75, 50, 37.5 and 8 Ohms are shown in Figure 12; as the resistance decreases, the voltage decreases and the function becomes more linear.

Refer to the documentation on specific chips for electrical characteristics of their analog nodes.

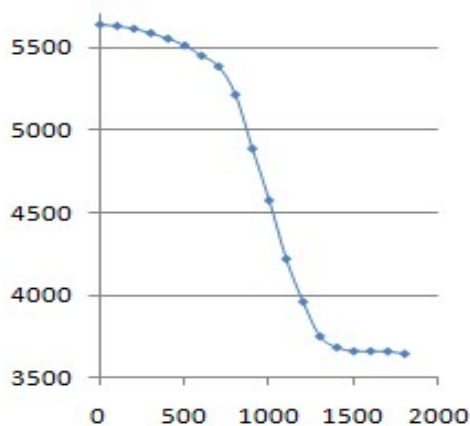


Figure 13 Typical ADC Transfer Function

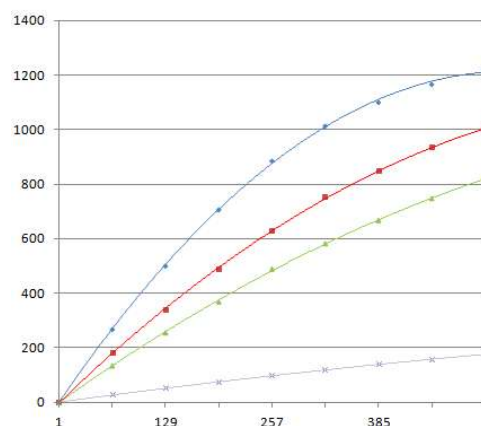


Figure 12 Typical DAC Transfer Function

4. Programming the F18

The environments of the arrayForth IDE for the x86-PC and pF/144 include a resident Assembler for the F18 instruction set. As usual, the F18 assembler is a "vocabulary engine" meaning that when the appropriate vocabulary is selected the F18 opcodes, directives, and user defined symbols are available. On the saneFORTH system this is vocabulary number 9 while on pF/144 it is number 5. The full Assembler environment also uses a special Interpreter to access user defined labels which are stored with the object code rather than in the local dictionary.

The assembler may be used in two distinct ways. The most common is to generate code to reside in and execute from a node's RAM or ROM. In this case code is laid down at consecutive addresses based on a location counter, and is assumed to execute at the addressing for which it was compiled, and in the node stated for things like the "warm" multiport execution address and for ROM content.

The second way is to generate an instruction word as a data item. The usual reason for doing this is to make an instruction word that will be used in port execution. In the past, it was often necessary to hand code such instructions and write them in hex; on the other hand, when it *was* possible to generate the instructions symbolically, the destination address checking for jumps and calls was based on memory location counter for the instruction stream in which the literal instruction was being stored, not on the port in which it would actually be executed. This could generate instructions that would not work as expected when executed in the port. This new method solves both problems. By default, no location counter is assumed and the assembler will generate jumps and calls that don't depend on the execution address of the instruction word. Directives may be used to specify an execution address (normally a port) for greater range of possible destinations at the expense of tailoring the instruction word for a particular port.

Other than the common essentials of Forth compilation technology, this environment includes the mechanism to compile code for hundreds of nodes, each of which has ROM that may be used by the application.

4.1 Object Code

The assembly process was monolithic in earlier versions of arrayForth. As of aF-3 we return to incremental assembly, with object code stored in a binary file area from which it may be accessed to load nodes or chips. This file area survives reboots so it is no longer necessary to reassemble all of the source code every time one needs any part of it.

A set of binary images, called **bins**, reside on pF/144 mass storage (serial disk or flash in pF) starting at block **Obin** (4800*3 or 14400 by default). Each bin is one BLOCK (1024 bytes long) and contains 256 bytes of RAM image (stored as 32-bit numbers), 256 of ROM, and a table of up to 50 labels for RAM and ROM code. One bin exists for each physical node, and there are additional virtual bins for code that may be used by utilities or placed in different nodes for different applications. The constant **nnc** gives the number of bins for which space is allocated. By typing **nnc** you will see on the stack the value 576 which is 144*4; the area is sized for programs using up to two chips, with an extra 288 nodes' worth of virtual bins into which object (for library code distributed by GreenArrays) may be stashed as noted later; see 8.2 Bin Assignments below.

At present, sF's Obin is at pF's Obin+2400 on the serial disk image. This is done to facilitate comparison of results between the two platforms. When we decide that is no longer necessary, this will cease and both systems will use a single set of object bins.

Because the label tables are persistent in object bins, a programmer with a twisted mind can create much confusion by defining primary Assembler directives or FORTH words as labels, giving the appearance of a broken system. If this has happened to you, either reassemble ROM for this node or fix it with the tools in the EXAMINE utility.

4.1.1 EXAMINE - Object Code Auditing

This minor utility for working with object code is useful during development. It can examine bin label tables and bin content in binary or dis-assembled, including comparison with reference binaries. Utilities are provided for initializing a set of bins and for moving quantities of bins.

Two sets of `nnc` bins are available to this utility. The first, starting at block `Obin`, is the object code produced by this machine's Assembler. The second, starting at `CFORG`, is a reference set produced by this or another system (such as `colorForth`). Facilities are provided for comparing these and for displaying differences.

-ALL erases all RAM and ROM object code for all `nnc` bins. Additional words `-CH0 -CH1 -XTR -XTR2` erase the four chip-size sections of the bin array.

?ROMS compares ROM for chips 1 and 0 with reference and displays an array of status indicators for all nodes with with "." indicating empty node in both images, "=" indicating identical ROM content and a highlighted "?" indicating disagreement.

?RAMS does the same for RAM object for pseudo-nodes 1600 thru 2317, chip 1 and chip 0.

USING (`nn`) selects a particular bin for examination.

.RAM dumps current RAM for selected bin, highlighting each word that differs from reference.

.CRAM dumps reference RAM for selected bin, highlighting each word that differs from current.

.ROM dumps current ROM for selected bin, highlighting each word that differs from reference.

.CROM dumps reference ROM for selected bin, highlighting each word that differs from current.

-ASM (`a n`) displays disassembly of the given address range (covering RAM and ROM) in the selected bin and annotating instruction words with labels. Differences from the reference are highlighted and the corresponding reference disassembly is shown in a second column.

.SYM displays the label table for the current bin.

COMMIT burns current RAM for the selected bin into the corresponding reference image. **Use carefully!**

In a normal development cycle where accumulated source changes are reconciled in the process of updating the backup, object can be audited in the same way. To set the current object as reference, use `DISKING` to copy `nnc` blocks from `Obin` to `CFORG`.

4.1.2 Getting Object Bins from `colorForth`

When converting an existing body of `colorForth` based code to `aF-3`, it is prudent to use the `colorForth` object as a reference. The following steps accomplish this:

1. In `colorForth`, **compile** all of the code of interest.
2. Say **named obj-cf** (or other name of your choice) to output file name.
3. Say **32768 4800 wback** to write all the bins to a 4800-block file of the above name. Only the first 864 blocks are relevant, but this simplifies file mapping into the `sF` system.
4. Move this file into the `pf` directory
5. On the `sF` system, in block 149 temporarily replace **OBJ-AF3** with your file's name, **FLUSH** and **RELOAD**.
6. See block 98. This is a utility for converting your incoming file, mapped as 11 UNIT, into the `aF-3` bin format as one of the sections of **OBJ-REF** which is mapped as 10 UNIT. To see a text directory of these sections, list block **10 UNIT 60** - Note that loading block 98 write enables 10 UNIT
7. Select one of the 600-block reference slots for your object and maintain the directory accordingly.
8. Load block 98 and say **n WAM** where **n** is the starting relative block (multiple of 600) for your data in the **OBJ-REF** file at 10 UNIT.
9. In block 149, restore the **OBJ-AF3** file name, **FLUSH** and **RELOAD**.
10. in `aF-3`, find the definition of `CFORG` and aim it at your new reference image. **FLUSH**, **RELOAD** and proceed.

4.1.3 Assembling Stock Code

While incremental assembly is generally a good thing, it does make object code persistent and as a result a lapse in attention or a simple mistake in numbering can deposit garbage in a bin with lasting effects.

Any time the integrity of the object bin array is in doubt you may re-populate it with known good content on either platform as follows:

11. Using the **DISKING** utility, verify the integrity of the aF-3 source code for tools and for all stock F18 code.
 1. Erase all bins by saying **EXAMINE LOAD -ALL .**
 2. Reassemble all GreenArrays code by saying **STOCK LOAD**
 3. Assemble any application code you have added.
 4. Verify object integrity with **EXAMINE LOAD ?RAMS ?ROMS**

4.2 Assembler Syntax and Semantics

F18 coding may be interspersed with high level polyFORTH coding; indeed, it may even be used to generate instruction words as literals within high level FORTH definitions. This requires clear and explicit bounding between F18 coding and host system compilation / interpretation regimes.

To assemble F18 code simply interpret Forth that includes one of the code bounding words `ASM[` or `A[` when necessary. The F18 assembler is resident in pF/144 and loaded as needed in saneFORTH.

4.2.1 Assembler State Variables

The Assembler's state is represented by five VARIABLES:

- IWD** 18-bit instruction word being built.
- 'SLOT** The next unused slot [0..3] of the instruction in **IWD** . If 4, further ops are forbidden (inline).
- 'IW** holds the F18 address at which the instruction in **IWD** will be stored. If negative, bits 10 and up are set, the low order bits [0..9] are the same as they are in **'IP** . In this case the instruction is being generated inline and will not be stored into a bin directly by the Assembler.
- 'IP** holds the address of the next word to store into target memory, corresponding with **p** register during execution. If negative, bits 10 and up are set, and the value of **p** is unknown for inline assembly. When an address is specified for inline assembly, the address is stored into the low order 10 bits of both **'IP** and **'IW**, and bits 10 and up of **'IP** are zero.
- 'CL** holds the slot number of call opcode in preceding word. Negative if there isn't one. Used in tail optimization. Second volatile cell is slot of last opcode stored.
- '##** Volatile flag set zero by **#** and true after parsing a number, a named literal, or a named call. When true these things perform their normal behaviors of generating literals or calls. When false, each of them leaves a number on the Assembler's stack.

4.2.2 Assembler Directives for Code Bounding

Source code compatibility between 32-bit (Win32) and 16-bit (pF/144) host platforms when generating code for an 18-bit computer is, fortunately, simple to assure. On both platforms anything being prepared to store into memory must be double precision. All other values are single precision on both platforms.

A special Forth interpreter provides the dictionary searching mechanism to allow access to **labels** defined in the current or in other nodes' **bins** in both Win32 and pF/144 implementations. In addition, this interpreter generates F18 literal references when naked numbers are encountered.

- ASM** Selects the Assembler's vocabulary without invoking the full environment, used when necessary in tool building.
- # (_ - n)** When running in the Assembler environment, as described below, most numbers occurring in the source code are intended to generate literals for the F18. When it is necessary to provide a number as an argument to one of the Assembler directives, preceding it by **#** will leave the number on the stack rather than generating a literal. **#** also conditions named values, such as the port names, to push values on the stack, and the same is true for named calls. This usage is shown where appropriate in the following examples.

Directives are used to control the Assembler's assumptions and behavior in generating inline instructions or bins for code. This section shows appropriate usage for each type of assembly. In the examples, curly brackets **{ }** surround optional words or phrases, and the vertical bar **|** should be read as "or but not both".

4.2.2.1 Inline Instructions

To build an F18 instruction and leave it on the stack, the following pattern is used:

```
A[ {# <port> =P} <opcodes> ]]
```

By default, the instruction will be generated with no foreknowledge of the value in **P** during execution. A port (or other) address may be specified using the **=P** directive. The stack value produced will be double precision. When used within F18 code the value may be laid down in the memory image being built using **,** while when used in high level Forth code the value may be laid down in host memory using **I,** . Each of these words is defined appropriately in each version of the Assembler to compensate for host cell size. The formal vocabulary is as follows:

- A[** Enters the Assembler environment, saving its state and setting up for generation of a single instruction word. Both interpretable and IMMEDIATE, usable in both FORTH and Assembler environments to generate instruction words in-line for use as literals or in building tables.
-]]** (-d) Exits that environment, returning a single instruction word (double precision) and restoring the Assembler's state as it was before **A[** was encountered. *When used in a high level FORTH definition, generates a 2LITERAL with the value of the instruction word.*
- =P** (a) Sets the assumed address in register **P** when this instruction is executed. May only be used within an inline instruction definition.
- ,** (d) lays the double instruction or other value down at the next available address in F18 memory, used when the phrase **A[...]]** is nested within F18 code (see below).
- I,** (d) lays the double instruction or other value down in host memory as a four-byte number, used when the phrase **A[...]]** occurs in high level FORTH code or data construction.

4.2.2.2 Building a bin

The bin mechanism allows ROM and RAM source code to be maintained and assembled separately. Ideally assemble the ROM code only once, and incrementally assemble application RAM code as needed. To build a bin of F18 code, the following general pattern is used:

```
ASM[ # ccyxxx NODE {# ccyxxx BIN} { {-SYM}ERS} | {-ROM} }  
# <addr> org always necessary  
<coding>  
FORTH ... ASM simple vocabulary switch  
]ASM <host code> ASM[ complete environment switch  
A[ ... ]] { , | lit } Generate instruction for  
table/lit  
{ >ROM } >BIN ]ASM
```

- ASM[** Enters the full Assembler environment for F18A *without initializing any of its state*. Visible in FORTH.
-]ASM** Exits that environment.

*Within **ASM[]ASM** the **ASM** vocabulary is selected and a special interpreter runs. As noted above in 4.2.2, this special interpreter recognizes the words in this section and by default generates opcodes in instruction words, literals and so on. To put numbers on the Assembler's stack it is necessary to use **#** because if you simply write numbers down they will be assembled as literals, and if you write a label it will be called. All numbers used with Assembler directives are of the width documented herein. Numbers to be assembled as literals **must** be written in double precision form.*

- NODE** (nn) Specifies which node's position, ROM and RAM to assume, and by default set the target bin to the same. Reads that node's bin into the local working image. Required.
- BIN** (nn) Overrides the target bin. Optional. *Bin assignments are currently controlled by GreenArrays; at this time we recommend you compile directly for the intended nodes and not use bin yourself.*
- SYM** Wipes ROM label table, used before **ERS** in nodes with many unnecessary ROM labels to recover space for application labels.
- ERS** Wipes the object memory and label table for the RAM portion of the working image. Does not touch ROM.
- ROM** Wipes the entire object memory and label tables in the working image.
- >ROM** Secures the current label table as ROM labels in the working image.
- >BIN** Writes the working image to the current target bin on mass storage. To write identical content to multiple bins, use **BIN >BIN** phrases repeatedly before **]ASM**.
- reclaim** This deprecated word was used in most colorForth code to prevent crashes due to filling the colorForth dictionary. Dictionary management in aF-3 uses conventional polyFORTH methods, applicable to any host code or data structures you might build. The host dictionary is not used for F18 labels in this system.

Later on, when loading code into nodes with the IDE "by hand" or when specifying boot conditions for tools such as the automated IDE loader or the stream generator, object code is identified by its *bin number*. By default that is simply the node number unless you have used **BIN** to stash the code elsewhere.

4.2.2.3 Special Interpreter Considerations

There is no problem with encapsulating multiple bins in a single source block, and in fact the source code distributed with aF-3 includes examples of doing this. However, the special interpreter activated by **ASM[** requires slight changes in common sF/pF practices.

4.2.2.3.1 Use of EXIT

If **ASM[** interprets an **EXIT** the block continues being processed by the normal FORTH interpreter. To achieve the usual effect of **EXIT** you must write it twice, as in **EXIT EXIT**.

4.2.2.3.2 Bins Needing more than 1 Block

This can be done in two ways, and the code distributed with aF-3 includes examples.

For the first method, see the code for Async Boot ROM. The first block loads the second within **ASM[]ASM** and the second block must begin with **ASM[** to activate the special interpreter. Note that the second block does not require any closing bracketing other than an explicit or implicit pair of **EXIT**.

For the second, see the Master DMA Nexus (bin 110) of the Ethernet NIC. The first block begins with all of the normal boilerplate to start a capsule's source, but has no closing bracketing. The second block begins with **ASM[** to resume using the special interpreter, and ends with the normal closing bracketing.

4.2.3 Location Counter

This is an incremental Assembler. Opcodes and literals are written into an image of target node memory as they are encountered; the only retroactive action it performs is to store into the destination fields of words containing forward referencing jumps (such as **if**) or calls (**leap**) when those forward references are resolved, or to change a call opcode to a jump when a call is followed by semicolon (tail optimization). The Assembler keeps track of the current position in target F18 memory with three variables documented earlier: **'IW**, **'SLOT** and **'IP**.

As opcodes are assembled, they are added to the instruction word being built at the address in 'IW and 'SLOT is maintained until the word is full or until the next opcode will not fit into the word. At that time the word being built is padded with . (nop) opcodes if necessary, and a new instruction word is started at the address in 'IP . This leaves 'IW pointing at the new instruction word and 'IP pointing at the following location in memory. The duality of 'IW and 'SLOT is necessary to support multiple instructions per word; the duality of 'IW and 'IP is necessary to support literals.

When literals are assembled, a @p opcode is generated and then the literal value is stored at 'IP , advancing 'IP . Alternatively you may write @p yourself and lay the following words down using , (comma), such as values calculated interpretively or instruction words generated by the assembler's inline nesting feature.

Jumping, calling, and memory operations address a word, not an opcode. When encountering a colon label, or any other assembler directive defining a place that may be addressed in memory, any code under construction is padded with . (nop) opcodes if necessary to align the location counter on a word boundary. This means that in absence of explicit control transfer opcodes, execution continues across alignment boundaries including the start of a colon labeled definition, a technique we have learned to use often.

The location counter 'IP is incremented in the same way the hardware increments P : The low order seven bits increment without changing the remaining bits of P . If you are generating code in RAM you will stay in RAM, wrapping its address space at x80 in terms of the value of P and also at x40 for actually addressing the memory image in the sense that the hardware ignores bit 6 of the address. The compiler will also generate code for ROM, in which case the wrapping points are at x100 and xC0. The P9 bit (x200) may be set as you wish to specify addressable destinations which will run in Extended Arithmetic Mode.

The location counter is managed using these words:

- .. forces word alignment; if an instruction word has been started, fills the rest of the word with nops. Equivalent to 1 <s1 .
- org** (a) forces word alignment then sets the compiler's location counter to a given address at which following code will be compiled into the current node's bin. 'IW and 'IP are initially equal but will separate after the first opcode has been compiled.
- #** Instructs the Assembler that the following number (or label, named literal, named call, or use if its [reference to a label in another bin]) should leave a number on the stack instead of assembling a literal or call as appropriate. For all but named literals the number will be single precision. If you wish, for example, to org to a given address you need to write a phrase like # x20 org or # joe org . See also 4.2.2 above.
- here** (-a) forces word alignment and returns the current aligned location.
- ,** (d) forces word alignment and lays the double instruction or other value down at the next available address in F18 memory, advancing location counter.
- +cy** forces word alignment then turns P9 on in the location counter. Places in memory subsequently defined will be run in Extended Arithmetic Mode if reached by jump, call, execute or return to those places.
- cy** forces word alignment then turns P9 off in the location counter.
- <s1** (n) Ensures that the next slot to be assembled will be *less than* the given number, forcing alignment in a new word if that is not the case. Used to solve problems with forward jumps or calls.

Slot assignment is a microscopic aspect of location counter management but is important to an F18 programmer for reasons such as optimizing forward references, aligning code on word boundaries for generation of literal instruction words to send another node through a com port, and the like. The words { . .. <s1 } are your main tools for managing slot allocation.

4.2.4 Control Structure Directives

These are used like those in classical Forth. The stack effects shown in square brackets reflect the host Assembler's stack; those shown in regular parens reflect the F18 stack at execution time.

4.2.4.1 Simple Forward Transfers

Forward transfer opcodes are assembled into the next available slot and may, unless you are controlling slot allocation yourself, be placed into slots 0, 1 or 2 with 10, 8 or 3-bit destination fields. The stack "handle" shown as **sa** for the unresolved forward transfer identifies both the location and the slot of the transfer opcode as well as the adjustment for **P** due to any preceding **@p** or **!p** opcodes in the same word. When **then** resolves a forward transfer, it will abort with error message "Range!" if the transfer is unable to reach the position at which **then** occurs without a larger destination field; when this occurs you must alter the code to resolve the problem.

if [-sa] If **T** is nonzero, program flow continues; otherwise jumps to matching **then** .
-if [-sa] If **T** is negative, program flow continues; otherwise jumps to matching **then** .
zif [-sa] If **R** is zero, pops the return stack and program flow continues; otherwise decrements **R** and jumps to matching **then** .
ahead [-sa] jumps to matching **then** .
leap [-sa] assembles a call to matching **then** .
then [sa] forces word alignment and resolves a forward transfer.

4.2.4.2 Count-controlled Looping

The F18 hardware supports looping under control of a count in **R** . The number in **R** is zero-based so the number of iterations such a loop makes is one greater than the initial value in **R** and that value will be zero during the last iteration of a loop. No forward transfers are used by these words and there are no issues with slots; all directives that generate backward transfers will pad the code if needed so that an opcode with the necessary size destination field may be assembled. The directives are as follow:

for [-a] (**n**) pushes **n** onto the return stack, forces word alignment and saves **here** to be used as a transfer destination by the directive that ends the loop. There are times when it is useful to decompose this directive's actions so that the pushing of the loop count and the start of the loop itself may be separated by such things as initialization code or a label. In this case you may write a phrase like **>r <other things> begin** .
next [a] ends a loop with conditional transfer to the address **a** . If **R** is zero when **next** is executed, the return stack is popped and program flow continues. Otherwise **R** is decremented by one and control is transferred to **a** .
unext [a] ends a micronext loop. Since the loop occurs entirely within a single instruction word, the address is superfluous; it is present only so that the form **<n> for ... unext** may be written. The micronext opcode may be compiled into any of the four slots.

4.2.4.3 Arbitrary Control Structures

As with ANS Forth, any desired control structure may be generated based on a few simple directives and flexible semantics; see the ANS Forth standard, or more to the point see the F18 code supplied with arrayForth, for many examples of composite control structures. The following directives are provided; the same stack notation (**a** for destinations and **sa** for handles to forward references) is employed here. If necessary you may code **SWAP** to affect the host Assembler's stack. New words introduced in aF-3 are shown in green.

begin [-a] forces word alignment and saves **here** to be used as a transfer destination.
while [x - sa x] equivalent to **if SWAP** . Typically used as a conditional exit from within a loop.

-while [x - sa x] equivalent to **-if SWAP** . Typically used as a conditional exit from within a loop.

until [a] If **T** is nonzero, program flow continues; otherwise jumps to a . Typically used as a conditional exit at the end of a loop.

-until [a] If **T** is negative, program flow continues; otherwise jumps to a . Used like **until** .

again [a] unconditionally jumps to a . The old colorForth spelling **end** may also be used.

repeat [sa a] unconditionally jumps to a and resolves the forward jump at sa . equivalent to **again then** .

else [sa - sa] jumps to matching then and resolves preceding forward transfer.

***next** [sa x - x] equivalent to **SWAP next** .

4.2.5 F18 Opcodes

The preferred opcode names, as shown in the *F18A Technology Reference*, each compile an opcode into a slot:

```
ex  @p @+ @b @ !p !+ !b !
+* 2* 2/ - + and or drop dup pop over a . push b! a!
```

The **call** opcode is compiled when an F18 label is referenced.

lit [d] generates a literal of the given value by inserting a **@p** opcode and laying the value down in memory.

alut [u] generates a literal of the given *unsigned* single precision value by inserting a **@p** opcode and laying the value down in memory.

<a valid number> encountered during assembly also generates a literal (*in which case it must be written as double precision!*) except when preceded by **#** in which case it leaves single or double precision on the Assembler stack as written.

S>D [n - d] converts a signed number to double precision, sometimes necessary with other directives.

Tail optimization is performed by **;** if immediately preceded by a **call**. In this case, the **call** is converted into a jump, conserving return stack space and leading to other useful techniques. Other jumps are generated by control structure directives, described later.

Four of the original opcode names assigned in colorForth, have proven to be poor human factors decisions because their names conflict with standard Forth usage and therefore create "programmer traps." In aF-3 we have deprecated the four opcodes shown in red above to eliminate these "traps" and renamed them as follows:

inv replaces **-** for a bitwise ones complement of T.

xor replaces **or** for a bitwise exclusive OR of S and T.

r> Replaces **pop** for moving a word from the return to the data stack.

>r replaces **push** for moving a word from the data to the return stack.

A block of code is provided to add the old colorForth names to the Assembler if you truly wish to use them, however it is not resident by default to save memory.

By default, unused slots are set to return (**;**) opcodes for object compatibility with colorForth (and with the code in ROM on the chip.) When the Assembler concludes that it must start a new instruction word within a code stream such that execution may continue from the preceding word into the new one, it must fill any unused slots in the preceding word with nops (**.**). You may do this yourself, for example to lay down an inline machine code literal not encapsulated by **A[** and **]** with the **..** directive that's described earlier.

4.2.6 Dictionary Labels

During assembly, a table of labels is built inside each bin image. Each label is stored as a counted string with maximum of the first seven characters actually saved for uniqueness. Each label has a 16-bit value. The following words manage this dictionary:

- equ** (n _) creates a new label whose name follows, assigning it the given value.
- :** (_) forces word alignment and defines a label at here as did red words in colorForth.
- its** (nn _ - n) normally assembles a `call` to the following label as defined in the given bin. However when preceded by `#` the *single precision* value is placed on the Assembler's stack.
- <valid label>** when encountered during assembly, a valid label defined in the working image assembles a `call` to that label. However when preceded by `#` the *single precision* value of the label is left on the stack.

Writing a valid <label> is equivalent to writing `# <label> call`.

4.2.7 Other Useful Words

Several additional resident definitions facilitate writing source code for F18s:

4.2.7.1 Named Literals

The following words, naming registers, normally assemble literals in the F18 instruction stream, however they simply leave their values (double precision) on the stack when preceded by `#`.

io right down left up data ldata

4.2.7.2 Named Literals for Cardinal Directions

When developing systems for our chips, the "floorplan" of assignments for nodes and clusters of nodes often needs to be changed, resulting in moving nodes or clusters from one place to another. When doing this, one nuisance is the need to change port names when moving a node(s) between odd and even rows or columns. In addition to being a nuisance it's an excellent way to introduce bugs. We address this at the levels of both the Assembler and of the Boot Descriptor Language.

The following words, naming registers by cardinal direction, normally assemble literals in the F18 instruction stream, however they simply leave their values (double precision) on the stack when preceded by `#`.

east south west north

4.2.7.3 Named Calls

Each of the 15 valid multiport addresses has a named word that normally assembles a `call` to that address. However when preceded by `#` they simply leave their *single precision* addresses on the stack:

---u --l- --lu -d-- -d-u -dl- -dlu

r--- r--u r-l- r-lu rd-- rd-u rdl- rdlu

await generates a call to the default multiport execute for a node based on its position in the array, also may be conditioned by `#` to return address on the stack.

4.3 Module Organization

A convenient convention has been adopted for organizing and packaging code in a modular way, from a single node to a cluster. This convention allows use of a single block number or name as a "handle" for the module. The first two or more blocks of the module have fixed functions at fixed offsets. We recommend that you apply these principles in packaging your own code. The SRAM Control Cluster Mark 1 may be studied as an example while reading the following sections; the constant **SRAM** is the number of the first block of that module.

4.3.1 Load Block

The first block in a module is a single block which when loaded will cause all of the source code needed by a module to be assembled. There may be arguments to this block. When completed the necessary object code will be stored in appropriate bins (which may simply be those belonging to the nodes programmed.) If by its design a module needs to export addresses or other identifiers, these will be available in the dictionary of that node *after it has been assembled*. *In a degenerate case this block may actually contain all of the module's source code*. See block **SRAM** for an example.

4.3.1.1 Identifier Scope

F18 assembly is done incrementally, which means that identifiers may not be "forward referenced" symbolically. Any identifier of any sort must be defined earlier in the assembly sequence than its references.

4.3.2 Boot Descriptors

The second block in a module (load block number plus one) defines the loading requirements for the module, using the high-level language described in section 6.1.1 below. This language lists the nodes that have to be loaded, indicates what code to load into their RAM, provides for initialization of registers and/or stacks, and provides a starting execution address for each node. The data are recorded in tables so that the order of declaration does not need to match the actual order in which the nodes are loaded. See block **SRAM 1+** for an example.

4.3.3 Residual Paths

In rare cases there may be modules that need to be loaded, activated, and used during a boot sequence; SRAM and SDRAM control clusters are examples of these. Once started it may be difficult or impossible to stop them without resetting the chip and, in the case of SDRAM, perhaps losing the data in the device. If an application's boot process has this sort of complication then the module in question should provide a new path for use by stream or IDE loaders that can reach and boot all nodes remaining in the chip that are not part of the module which is running and hence now "in the way." See block **srpm 2+** for an example, showing suitable paths for boot nodes 708 (async serial) and 705 (SPI flash.)

4.3.4 Organization of Larger Projects

The appropriate structure for a larger program depends on its size and intended use. The load block should still be first. Boot descriptors should start in the second block but may require several. If IDE operation is appropriate after the program has been loaded, an IDE personalization with path(s) adjusted to access the remainder of the chip may be useful. Scripts may be necessary to specify boot stream generation, IDE loading, and/or **softsim** setup as appropriate. Not all of these elements are appropriate for every application, but if they are small you might wish to place them in this area after the boot stream and residual path allocations, and before the actual source code. These are merely suggestions and the recommendations may change as the system evolves.

4.4 Methods of Loading Code

After all the desired code has been compiled into object bins, it may be loaded for execution or simulation. There are presently several basic ways in which this may be done:

Manual, interactive loading into the real chip with External or Internal IDE: Insert code into RAM, do any necessary initialization manually, and call routines to observe behavior, test boundary conditions, look for side effects and so on. It is impossible to over-emphasize the importance of this practice. Simple, straightforward unit testing is a fundamental entitlement of a Forth programmer and its use dramatically simplifies debugging later on. Don't deny yourself this advantage by combining a cluster of nodes full of untested code unless you enjoy coping with serious problems either immediately or some day in the future.

Automated loading into softsim: Initialize nodes as directed by Boot Descriptors.

Automated loading into the real chip using boot streams: Use the **STREAMER** utility to construct a boot stream as directed by Boot Descriptors, then present the resulting stream to a boot node or inject it into the chip using the Snorkel.

Fully automated loading into the real chip upon RESET- signal: Use the **STREAMER** utility to construct a boot stream as directed by Boot Descriptors, then burn it into the front of SPI Flash.

Each of the automated methods uses the common Boot Descriptor Language to define what code, if any, shall be loaded and what additional initialization shall be performed before starting each node.

Which methods are appropriate depends on which host system you are running on and what connection you have to the target system, if any. sF may use one or two serial COM ports connected to one or two ("host" and "target") chips. pF/144 may use its 1.8V async interface from node 708 (requires jumpers and cables), or its 1.8V sync interface from node 300 either to the target chip on the same eval board (default jumpers), or to another chip (requires jumpers and cables). pF/144 uses node 500 to drive the reset pin of the chip and this also requires attention to jumper and cable depending on the target chip.

pF/144 may also inject streams into the chip it's running on via the Snorkel in node 207, and by building a bridge these streams may continue into another chip. For example, if you're running a basic polyFORTH system that doesn't have the Ethernet NIC nor the external clock code booted into the chip's nodes, you can add this by loading block 1583. This is a simple example of using STREAMER to build a boot stream that loads the Ethernet and crystal support, then injects that stream into the chip through the snorkel in node 207.

Boot Stream Load Blocks, in an earlier section, identify load blocks that may be used from saneFORTH for serial booting of chips, or from polyFORTH for building boot streams for insertion into flash or for interactive injection via node 207. These blocks are present in both systems and are adapted as needed for use in each. The serial booting mechanism from polyFORTH is not yet supported.

5. Interactive Testing

Interactivity is a critical, cardinal virtue of Forth as a programming system. Turning Forth into a batch mode programming system would be a travesty, sacrificing one of its most potent properties and setting the clock back nearly half a century. When using Forth, a programmer becomes accustomed to having her fingers literally on the fabric of the computer, directly manipulating its memory, registers, and other resources; exercising hardware and software without necessarily having to write (and debug) software simply to exercise those things.

The IDE was implemented as a natural and obvious extension of the umbilical methods used with embedded Forth systems for many decades. The method used is as non-invasive of hardware and software as is practical in this architecture, requiring no RAM or ROM in a node being tested and touching its registers as lightly as feasible.

With aF-3 there are two different environments. External IDE allows a host system to manipulate a target chip using synchronous or asynchronous serial interfaces as appropriate; this environment has been in use with arrayForth since the dawn of GreenArrays. Internal IDE is new with aF-3 and employs the Snorkel/Ganglia mechanism to permit a pF/144 system running on a G144A12 chip to interact in analogous ways with other nodes on the same chip as pF/144 is running on (*even some nodes that are part of the pF Virtual Machine!*) and, if the chip is bridged to another, on a second chip as well. There is a technical App Note AN019 on the IDE internals. This section covers the use of both environments.

5.1 External IDE

This utility is implemented for both sF and pF/144.

5.1.1 Terminology

Debugging is done on one or more *target nodes*, using an umbilical connection which will involve one or more intermediate nodes of one to three kinds (*root*, *wire*, and *end*), depending on the physical location of the target node within the chip and on the *path* taken to reach it. For IDE into the asynchronous interfaces of a GA144 including either of the chips on an EVB001 board, the root node for either chip will always be 708 because that is the one and only asynch boot node on a G144A12. The asynch IDE may thus be used to debug code in any node *except* 708.

The inner load block for the asynchronous IDE is **SER** but this is intended to be used as a factor of other utilities. To work with the host chip, say **HOST LOAD** and to work with the target chip say **TARGET LOAD** after remembering to edit **A-COM A-BPS C-COM** and **C-BPS** (in CONFIG and SERIAL, or in config.txt if you are using a project directory) to define the correct COM ports and line speeds.

5.1.2 Using the External IDE

1. Assemble the necessary code into bins and make any necessary connections to the target system. Set serial port numbers as needed. **sF ONLY:** Connect at least one COM port (usually high speed FTDI) to the target system. If this is a new port, configure it correctly as described in the Appendix applicable to your platform. Edit the main configuration blocks, CONFIG and SERIAL, or the config.txt file if you are in a project, to reflect this port number as host or target chip, and select a baud rate that works for the electrical interface in use. Repeat this process if you are using two connections.
2. Examine and, if necessary, change the path lists to make sure you can reach all the places you need to. See the definitions of 0PA 1PA 2PA after the block named SER for examples.
3. Make sure nothing else in the windows environment (including dialog boxes) has any open handles for these device(s).
4. To work with the host chip, say **HOST LOAD** on either platform. **sF ONLY:** To work with the target chip say **TARGET LOAD**.

5. Say **TALK** to reset the target chip (may be prevented) and download root node talker code into the chip. ***On some target boards, you may need to cycle power or press a reset button before this step.*** If the serial port is already set up and you do not wish to reset the chip, use **Talk** instead.
6. Say **.PTH** for a concise display of IDE state. You should at this point have three defined paths, one to each of the nodes immediately adjacent to the root node, for example:

```
.PTH
 2  709    1  709
 1  608    1  608
 0  707    1  707 <--- SELECTED
```

The first column is path number; the second shows which direction it goes (first node away from the root). Third column shows the number of COM port boundaries between the root and the target node, and the fourth column is the target node to which the path is currently connected.

7. To see more about the target node use **SEE** which displays the path state as well as the stack and RAM in the target node.
8. Define, tear down, and select paths as you wish, and operate on the nodes in question using the words described below. *Note: If you wish to prove that all other bootable nodes are completely idle, begin your activities by saying **2 <root> HOOK** (for example, **2 708 HOOK** when using async IDE) and then **2 -HOOK** to tear that path down. The default path 2 can reach all 143 nodes accessible to the IDE.*
9. Operate on the selected target node as you wish using the basic or enhanced vocabulary.
10. If you wish to change the code you are downloading into any node(s), change the F18 source and load it to update the object bin(s). IDE state is not affected by normal assembly. Any paths you had wired up, and whichever you had most recently selected, will still be in effect as you can see by displaying **PANEL** again. *Assembly does not communicate with the chip under test.*
11. When you are finished you may depart the IDE like any other FORTH utility by typing **EMPTY**.

5.1.3 External IDE Vocabulary

Words come in two classes: Those which wire, rip and select paths, and those that operate on the target node.

5.1.3.1 Path Routing Control

To see how the path routing is currently set up, say **.PTH**. Node numbers used with these words are always in the cyxx form.

PATH (i) Selects path *i* (0, 1, or 2) so that all subsequent target operations will apply to the target node for that path. *This is necessary when more than one path leads to the same target node.*

NODE (nn) Selects whichever path presently has node *n* as its target, if any. Note that it is possible to set up more than one path to different ports of the same node; if you have done this, **PATH** will permit you to select the desired port. Leaves a default path selected if none of them targets the node given.

HOOK (i nn) Wires path *i* to target node *n* after ripping out any existing wiring for that path, leaving path *i* selected. If the node in question is not accessible in the route list for that path, leaves the path set for the appropriate adjacent node as target.

-HOOK (i) Forces ripping out of any wiring for path *i*. Each node that had previously been part of this path is normally left in its default **warm** state (multiport execute); the target node *is not affected*.

Path Node Hook -Hook These words do the same things, but silently.

PTB (i-a) an array of starting addresses indexed by path number. To adopt a new path, store its address into this table at a time when that path index has been unhooked.

5.1.3.2 Target Operations

Each of these operations applies to the target node of the currently selected path.

UPD retrieves all ten words of the data stack into the local array **STACK** which is indexed (T, S, and the eight elements in the F18 stack, first element being the one that would next be popped into S). Displays the stack conditionally (using **?STK**).

.STK displays the contents of **STACK** in two rows, in the current **BASE**. First row is the deepest four elements of the F18 stack, followed by the shallower four elements, then S and T. The shallowest element is immediately left of S; “deeper” moves to the left on that row, then to the right on the row above, with the deepest element on the right side of the top row. Thus, the eight words of the stack array seem to move circularly. This is isomorphic with the stack array and makes its behavior clearer.

```
.STK                vv--Next Push from S
2AAAA 2AAAA      15      15      (S)      (T)
2AAAA 2AAAA 2AAAA 2ABAA 15      15
                ^^--Next Pop to S
```

?STK displays the stack as does **.STK** if **?MUTE** is zero.

?RAM displays all of RAM from the selected node (in the current **BASE**).

?ROM displays all of ROM from the selected node (in the current **BASE**).

-ASM (a n) disassembles **n** words of code starting at **a** in ROM or RAM. Labels are integrated from whatever bin is current.

LIT (d) removes a double number from the host computer's stack, pushing it onto the data stack of the target node and displaying the updated stack if not muted.

R@ (a-d) reads RAM, ROM, register or port word at **a** in the target node, pushing the data read, **d** onto the host computer's stack. *In this release, uses and does not restore register **a** in the target node.*

R! (d a) writes a double number from the host computer's stack into target memory (RAM, register, port) word at address **a**. *In this release, uses and does not restore register **a** in the target node.*

RINS (d) Executes a given instruction word in the port of the target node. Usually built with **A[.. ;]**

CALL (a) calls, from the port, the code at address **a** in the target. If that code returns to the port or re-establishes the normal rest state of the target node, interactive use may continue once the code completes. No interlocking is done so this method may also be used to start application processing which will not admit to further port execution access. Interaction with a node actively running an application may also be arranged but at the cost of periodically polling, as described below.

BOOT (a n nn) loads code into the target node, starting at address **a** in both target and bin for **n** words, from the current binary output area identified by **nn**. This need not be the same node as the target.

FOCUS forces the target to call only the port on which the current IDE path is talking to it. This remains in effect until the target is directed to execute elsewhere.

VIRGIN forces the target to call its default multiport execution address to un-do the effects of **FOCUS**.

IO DATA UP DOWN LDATA LEFT RIGHT place chip port addresses on the host's stack.

RB! @B !B RA@ RA! @A !A @+ !+ R+ R+* R2* R2/ RINV RAND ROR RDROP RDUP ROVER Execute single F18 instructions in target, using target's stack, and update stack display in panel. Note: These functions may be omitted in favor of **A[opcodes ;] RINS**

5.1.4 Advanced External IDE Uses

When ripping a path out, all wire and end nodes (if any) are left in their default multiport executes. This is necessary so that, for example, paths may be crossed so long as they are not used concurrently to cause conflicts. There are occasions, such as when starting nodes that send unsolicited code or data to other nodes, when this is not appropriate. In those cases the following two functions are useful:

UNFOC Conditions -hook to work as stated above..

FOC Conditions -hook to leave every wire and end node focused back toward the root. This leaves the path completely impenetrable to anyone other than the root node, and further hooking and starting of nodes will be necessary if that path is ever to be permeable again.

The IDE is factored such that it may become a tool of other utilities. Examples are the **SELFTEST** and **AUTOTEST** routines, the IDE boot and flash burning tools used for polyFORTH, and even the **HOST** and **TARGET** load blocks. Until these conventions are fully documented here, please read the code just identified to see how it is done. In the meanwhile here are general points:

1. **HOST** begins by loading the necessary code, and then sets up the communication environment.
2. If the IDE is to be used in an automated fashion, it's usually correct to set **?MUTE** nonzero. This will prevent chatty displays of the routing every time it's changed, and will allow use of remote instructions without updating the stack display gratuitously.

5.1.5 Configuring IDE for Target Environments

As shipped, the IDE is configured for the environment of the EVB002 Evaluation Board. This will do for many other circuits using our chips, however the target environment may require attention to several things. Most of these have to do with devices connected to GA144 I/O pins or to code being loaded from flash.

- **Flash Code:** In some cases it is necessary to enable flash booting even when planning to work with the chip using the IDE. For example, some I/O initialization may be always necessary, such as switching the 18-bit parallel Address and Data buses from output to input mode, or placing other pins in states other than their default weak pull-down. The existence of such code may require adjustment of high timing in the IDE's definition of **RESET** (10 ms as delivered, to accommodate one such board we use in development.) Additionally, if a watchdog circuit is active in the target environment, this flash boot may need to load code into at least one node to keep that circuit from resetting the chip; this will require changing IDE paths from the defaults. Finally, in some cases a more extensive amount of the chip or chips is booted but we still want to keep the external IDE usable on nodes not (yet) programmed. In this case more extensive IDE path changes will be required. In either case, it may prove difficult to devise single IDE paths that can reach all nodes one might wish to program.
- **Connected Devices:** If our output GPIO pins are high at the time of reset, the amount of time it takes for these to be restored to low state after the GA144 is reset can be rather long if the capacitance on the pin is high. Our inherent pin capacitance is on the order of 2.8 pF but that is without connection to a PCB or to driven device(s). The output circuit is typically an RC first order lag, with the weak pull-down transistor looking like a 50k resistor. The time constant of this circuit is on the order of 500 ns per 10 pF. This is more important for on-board reset circuits than it is for the external IDE, but it should still be taken into consideration if extremely high capacitances are involved. Another property with similar results might be the time required by a device to restore its outputs to resting conditions upon removal of an enable or other signal due to resetting the GA144. In all such cases the **RESET** low time must be long enough to ensure that code in the chip, particularly boot code, does not awaken at end reset and see a pin high when it should be low. The default low time in the IDE's definition of **RESET** is 5 ms as delivered.

5.1.6 Working with Two Chips using External IDE

The IDE may be used to interactively debug software on both chips of the Evaluation Board as though they were a single chip with 288 nodes (two of which are invisible), using just the serial interface for the host chip.

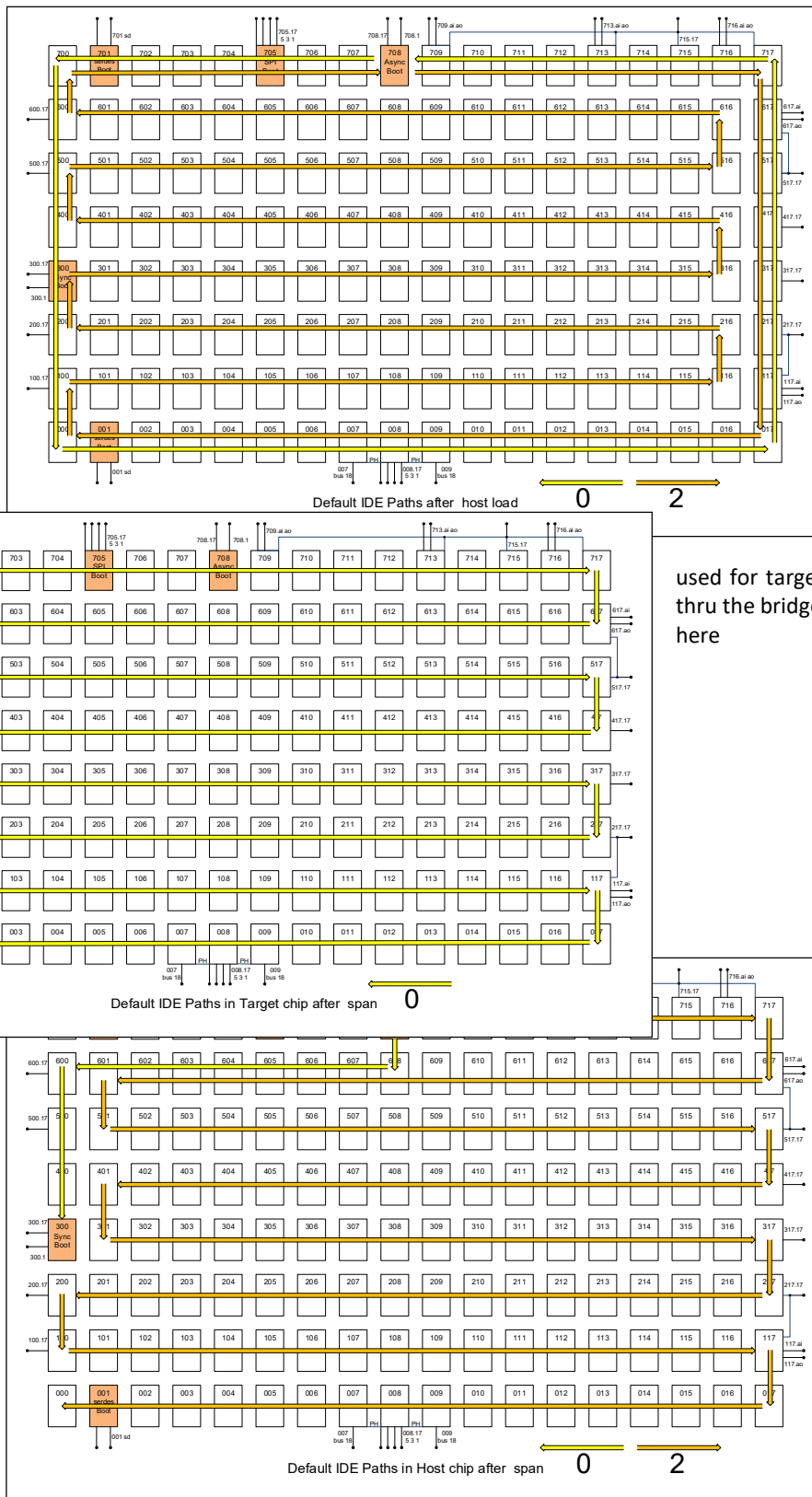
BRIDGE LOAD compiles a host IDE version capable of operating on both chips. Initially its environment and function is identical with **HOST** and you should establish connection with the host chip using **TALK** if it isn't already established.

SPAN extends the bridge IDE to encompass both chips. In order to successfully invoke **SPAN** the edge nodes of the host chip starting with 707, proceeding to the left to 700, and downward to and including 300 must be accessible for programming; it is typically used immediately after loading **BRIDGE** and establishing host connection unless something such as polyFORTH needs to be set up on the host chip first. **SPAN** resets the Target chip and programs node 300 on each chip as a transparent bridge for carrying port communications between them, using 2-wire synchronous communications. New default paths 0 and 2 are set up to cover both chips. Thereafter, nodes 300 are dedicated to this purpose until the chips are reset.

With the port bridge built, the **up** ports of node 400 on each chip are logically connected as though they were a simple COM port. Port read/write communications, such as IDE, are basically transparent across this connection except that data transfers take 100 or more times longer. The current version of the bridge supports flow control, so unlike the old version node 400 may determine whether the bridge has data for our chip or is ready to send a word to the other chip by examining **io**: If the bridge is sending us a word, node 400 can see **up** writing, and after we write to up from node 400 we will not see **up** reading again until the word was accepted by 400 in the other chip. *However, you may not infer by seeing **up** reading that node 400 on the other chip is at this time reading its **up** port. Any programming method that depends upon such awareness will have to be implemented in some other way.*

The default paths after **SPAN**, shown below, facilitate full access to the Target chip with or without the polyFORTH virtual machine present on the Host chip. Extended node numbering is supported in the form *cyxxx* where *c* is zero-relative chip number; thus nodes 000 through 717 are on the Host chip, while nodes 10000 through 10717 are on the Target chip. The IDE is aware that a connection from 400 to 10400 exists through their up ports. ***You may only have one path hooked through the bridge at any time! Attempting to violate this rule will simply hang.***

5.1.7 Default External IDE Paths



After loading HOST, BRIDGE or TARGET these paths exist in the chip to which the serial connection has been made. Path 1 is available for general use at that time.

After SPAN path 0 is generally used for target chip, starting on host and passing thru the bridge to cover the entire target as shown here

... and path 2 is generally used for the host chip. Path 1 is available for general use. Path 0 by default avoids passing through the top row of nodes on the host because in polyFORTH environment node 705 is programmed for SPI flash operations.

Because paths 0 and 2 share nodes one should be unhooked before the other is hooked through those nodes. The port bridge only requires special code in nodes 300 and 10300.

5.2 Internal IDE

5.2.1 Terminology

5.2.2 Using the Internal IDE

Internal IDE Vocabulary

6. Preparing Boot Streams

While the external IDE may be used to load and interact with F18 code in any chip for hardware or software testing without requiring any other hardware or initialization on the part of the chip, higher level modules such as the polyFORTH Virtual Machine may require additional hardware and the loading and initialization of many nodes as well as perhaps external SRAM in a practically simultaneous manner. This is best done by generating a **boot stream** to do all of that.

Unless equipped with special ROM or booted using SERDES, our chips must be booted after reset by having a suitable **boot stream** made available to one of its **boot nodes** (such as async serial, 2-wire synchronous serial, or SPI flash nodes that are enabled for boot after reset.) A boot stream consists of one or more **boot frames**, which are data structures defined and processed by the boot nodes of our chips. Each boot frame contains zero or more words of data, a starting memory or port address at which the data are consecutively written, and a jump address to which control is transferred by the boot node after the frame has been processed. Every boot node defines a **concatenation address** to which a boot frame may jump to process another frame. After reset, a chip may boot itself from a slave device such as SPI flash memory, or it may wait to receive boot stream(s) on one or more of its enabled interfaces. For example, a daisy chain of subsidiary chips may be booted by a master chip using frames stored in the master's mass storage or read from its boot device (such as an SPI flash). As another example, the external IDE works by transmitting boot frames into the chip under test. arrayForth hosts can build and transmit streams of boot frames into a target chip for efficient booting (the old IDE based boot mechanism has been deprecated in arrayForth-3.)

6.1 The Streamer Utility

This utility is a set of tools for generating and packaging **boot streams** that consist of one or more **frames**. In a boot stream, each frame has a **header** to be interpreted by a **boot node**, and will end either by returning to a simple function in the boot node such as its **warm cold** or its appropriate **concatenation address**; in some circumstances the boot node will be given code to execute, which may read additional data from the boot stream and dispose of it in some way (for example, copying the polyFORTH nucleus into external SRAM) before completing its job and returning to such a place in the boot node, perhaps with a new **starting address for the next frame** in the case of SPI flash. Streams may also be generated for insertion into a node attached to the Snorkel, in which case no boot headers are necessary and special methods must be used if that first node is itself to be programmed.

The Streamer operates in two phases. In the first, it composes the entire stream as instructed, one 18-bit word per 32-bit cell in memory (in pF/144 we do this using unallocated space in the nearly 1 Megaword of extended memory.) In the second phase, the boot stream is converted into a form suitable for whatever medium is being used (for SPI it is a stream of 18-bit values expressed as a succession of bytes; for async serial each word is inverted and shifted into a 3-byte form equipped for auto-baud of each word. For delivery through the Snorkel, it is left unchanged.)

A minimal use of the Streamer consists of the following elements:

STREAMER LOAD	Compile the utility
nn STREAM[	Specify root node and clear
{a COURSE}	Override default path for that node
FRAME[	Begins a frame suitably for root node
boot description ---	see 6.1.1, Boot Descriptor Language
(BDL)	
]FRAME	Ends a frame suitably for root node
and,	
resolves	if a forward reference was made,
location.	it at the current aligned flash
]STREAM	Completes a stream and packs it as
needed	

Here is the full STREAMER vocabulary, followed by the Boot Descriptor Language:

STREAM[(nn) If usage from root node nn has been defined, selects the header formulation and final packaging suitable for that node as well as a default path whose address is returned by **ENTIRE** after **STREAM[** has been used. Clears the entire stream buffer.

The node numbers defined for this use are at the time of this writing **207, 708, 300** and **705** . For node 708 we use asynch serial header and 3-byte asynch word packing. For node 300 we use synchronous serial header and don't pack the stream. For node 207 we generate no headers and do no packing for Snorkel injection into the chip. And for node 705 we use SPI flash headers, compress each 8 18-bit words into 18 bytes (9 16-bit words), and set special end-frame behavior for forward references because at the end of loading each stream node 705 must be instructed to read the next stream starting at a byte boundary.

COURSE (a) Overrides the default path selected by root to be used as **ENTIRE** at any time. In a multi-frame stream it will often be necessary to change the path, for example because an earlier frame some of the chip's nodes have been programmed. It is the nature of boot streams that the first node loaded will be the last one in the path. The first node in a path *must* be the number of the node from which it originates. The address given must be the tick of an **ARRAY** holding a sequence of nn numbers ending with -1. Default paths for the standard root nodes are named df708 thru df207 .

FRAME[begins generation of a frame. All descriptor tables are initialized with their default settings. The root node is either not programmable at all (node 207) or partly programmable (memory settings and /P are the only valid operations on a boot node itself)

]FRAME Ends a frame. Calculates the frame length and generates the appropriate frame header if any, followed by all port and memory pumps to load the nodes in the current **COURSE** .

/ROOT (jmp a n bin) is used outside of **FRAME[]FRAME** encapsulation to generate a boot frame for the root node, loading that node with n cells from the given bin and jumping to the address jmp .

FORWARD is used immediately after **/ROOT** when generating a stream for SPI flash to indicate that the program just loaded will access flash before resuming stream processing. When this is done the address register in the flash loses synchronization with bit stream reading. In order to start up correctly again, the stream is padded to the next zero modulo 8 18-bit word index in the overall flash image, and the code loaded in the root node is patched to provide the absolute byte address for continuation in a standard manner. See definition of **FORWARD** and the source code it is used with.

]STREAM ends a stream. Converts the stream in place to the appropriate form for the intended node of origin.

.STREAM dumps the current stream, which must be in unpacked numeric form.

?STREAM audits an unpacked stream against a reference stream, see below.

?FLASH audits a packed flash stream against a reference stream, see below.

STREAM (- a n) **sF ONLY** returns origin and length in octets of the converted stream. Normally used within other tools.

STREAM (- da n) **pF/144 ONLY** returns origin and length in octets of the converted stream. Normally used within other tools.

6.1.1 Boot Descriptor Language (BDL)

Automated loading *touches* all nodes in a defined path starting at the relevant root node. Automated **SOFTSIM** loading touches all nodes in both chips. By default, a stream does as little as possible to each node touched. One item is by necessity pushed onto each stack, and register **A** is altered. Register **P** is set to the appropriate IDLE multiport address for that node, and **B** is set to the address of **IO** for all nodes not otherwise loaded. BDL is used to specify non-default treatments using statements that start with the word **+NODE** and continue with phrases describing the needed initialization of memory and/or registers in the node. Nodes may appear in any order since these statements are simply filling tables for later use. For the same reason, **+NODE** statements are cumulative in that one **100 +NODE** phrase may set the memory loading and starting address for node 100 while another **100 +NODE** phrase interpreted later may

override the starting address specified earlier. The following words constitute the BDL. Their use in context is described here and in the chapter on **SOFTSIM**; examples may be found in the distributed source code.

+NODE (nn) Selects table entries for node number *nn*, in *cyxx* notation. The values in the table entries are not changed by **+NODE** and so their values will be default unless a previous **+NODE** phrase has been interpreted for the same node.

/RAM (bin) Loads all of RAM from the given *bin* in *cyxx* notation. *By default nothing is loaded.* Partial loads may be described using the following words. Up to three RAM load descriptors may be specified for each node. Such loads are applied to the node in the order encountered in the boot description, so that if there are overlaps the last descriptor will overwrite earlier ones. When a RAM load of all 64 words is specified, any previously declared RAM loads are deleted and will not appear in the stream. Thus it is possible to preset a default background, such as of Ganglia, and override that for nodes that won't be able to serve as Ganglia, without extending stream length.

/SOME (s d n bin) Loads part of RAM, *n* words from the given *bin*, starting at address *s* in the *bin*, are loaded into the node's RAM at address *d*.

/PART (a n bin) Loads part of RAM, equivalent to *a a n /SOME*.

/B (d) Specifies an initial value for register *B*. *By default B is set to IO.*

/A (d) Specifies an initial value for register *A*. *By default a port call instruction is left in A.*

/IO (d) Specifies a value to be loaded into the *IO* register. *By default IO is not altered.*

/STACK (<n double values> n) Specifies up to ten values to be pushed onto the data stack, with the rightmost value on top. For example *30 20 10 3 /stack* produces the same effect as though a program had executed code *30 20 10*.

/RSTACK (<n double values> n) Specifies up to nine values to be pushed onto the return stack, with the rightmost value on top. For example *30 20 10 3 /stack* produces the same effect as though a program had executed code *30 20 10*.

/P (a) Specifies an initial value for register *P*. Default value is *xA9* which is the routine *warm* in every node's ROM. When the node is started, *warm* jumps to the appropriate multiport execute for the node's position in the array. If you wish to leave the boot routine enabled in a boot node that is touched by an automated loading procedure, specify its *P* value as *xAA* as is done by *reset* for such nodes

!ND (nn) Macro to load node *nn* with code from same *bin*.

+ND (a. b. p nn) Macro to load node *nn* with code from same *bin* and initialize registers.

ITS (nn _ - a) Returns the value of the label whose name follows in *bin nn*.

RIGHT LEFT UP DOWN IO LDATA RDATA Register names returning double addresses in I/O space.

EAST WEST NORTH SOUTH Register names like the above, but leading in cardinal directions for the node selected by **+NODE**.

6.1.1.1 BDL Macros

Several higher level BDL functions are also provided:

/RINF initializes return stack with all -1s. Used to support infinite <op> <op> unext unext loops.

+wire (s d nn) places a perfect 1-word wire in node nn with source and destination ports s and d respectively.

+dly (s d nn) Sets node nn up as a delay line from ports s to d .

6.1.2 Stream Structure

The components used for building frame content differ from those used in the External IDE. This section includes information for calculating worst case frame length. For calculations based on node quantity, we assume a bridged pair of chips containing a total of 288 nodes, two of which (300 and 10300) are inaccessible because they are the bridge, and the boot node itself which is loaded in a separate frame; so the frame will visit a new of 285 nodes in the full, 2-chip path.

A frame begins with a 3-word header unless it isn't needed. The first node receiving data via the frame body is given a focusing call that must be present in the stream; subsequent nodes get their focusing calls from the pump in the preceding node. So we begin with 1 or 4 words and add to it modules of the following kinds.

6.1.2.1 Port Pumps

A frame begins with one port pump for each node except the last in the path. The port pump is still five words but uses **A** rather than **B** to point to the port at which the pump is directed; thus the port pump clobbers A, and it also has to push one word on each stack. For n nodes in path after the root, n-1 pumps are required; for two chips n-1 is 284 so 1420 words of pumps add to the 4 above for 1424. Structure within the stream:

04DAF	@p dup a! @p (preceded by focusing call)
12xxx	Literal, the focusing call given to next node
lng-1	Literal, number of words following pump -1
2FAB2	>r !
05A72	begin @p ! unext

Following the sequence of port pumps, there is a Memory Load module and a Post-Load Initialization module for each node beginning with the last node in the path and proceeding backward along the path to and including the first node after the root. When a node given a port pump is finished with pumping, the next thing in the stream will be its memory load(s) (if any) and initialization (at least of P and IO.)

6.1.2.2 Memory Loads

Each memory pump is five words long followed by the text to be loaded. In the typical case there is one memory pump and 64 words of text for a total of 69 words. Complicating this, we permit combining up to three memory loads for a node, in which case one might expect still a max of 64 words of text but with two additional pumps, bringing the total to 79 words. 285 of these would be 22515 words, for a total thus far of 22939 although a smaller number is more common because normally there is only one memory load in effect for each node. Structure:

04A12	@p a! @p
<adr>	Literal, start address of this RAM load
lng-1	Literal, number of words following -1
2E9B2	>r
05872	begin @p !+ unext

It is possible to describe an even worse case in which there would be three fully overlapping 64-word memory loads for each node, but this is not worth considering. We do make provision for all nodes to be initially loaded as a background default by such things as the Ganglia, but to avoid letting this run up the size of the frame we check in the RAM load specifiers for full 64-words descriptors and if one is encountered all previous RAM load descriptors for that node are deleted so that they will not be present in the frame.

6.1.2.3 Post-Load Initializations

Initializing IO costs 4 words; A and B cost 2 words each and P costs 1. Initialization of each stack takes 2 words per value. We could fight for a reduction on the stack initializations but the benefit would appear to be marginal. The initialization components are structured as follow:

Set IO	04BB2	@p b!
	0015D	Literal, address of IO
	05BB2	@p !b
	value	Literal
Set A	04AB2	@p a!
	value	Literal
Set B	04BB2	@p b!
	value	Literal
Push R	048B2	@p >r
	value	Literal
Push S	049B2	@p
	value	Literal
Set P	10xxx	Jump to start address

P is (and must be) always set, so the minimum module is 1 word; for consistency with colorForth, we also always initialize B, to IO if nothing else is specified in the BDL. To initialize everything possible we have 9 words for the discrete registers and 38 words for all of the stacks, for a total of 47 words worst case and 13395 for two chips. This brings us to 36334 words so far for a practical worst case 2-chip frame.

That number pushed us across a 64k byte boundary and so we have chosen to reserve the first 128 kB of flash media for boot stream text.

6.1.2.4 Root Node Programming

The root node may simply be programmed as the last step in a complete boot stream. Earlier in the stream it may be necessary to program the root node to do other things such as for example speeding up the flash clock timing, or copying data from flash to an external SRAM as is done in booting eForth or polyFORTH.

Such intermediate programs for the root may be generated using /ROOT which does not employ BDL, or included in the BDL for a stream in which case the streamer generates two frames for the boot path (one for all nodes but the root, and one for the root itself.) When such a program will be accessing other flash during its operation and then resuming stream processing, the word FORWARD should be used after]FRAME in a flash boot stream to pad the stream to the next even byte boundary in flash, and to patch the root node program for a forward reference to the padded restart address.

Only one memory load may be specified for a root node, and it must have the same source and destination offsets if the load is specified by /SOME. Although it is possible to initialize more than RAM and P in a root node, this process would be complex and we have not built any particular such mechanism into the BDL. None of this is relevant for node 207, but it is true for any real boot node.

6.1.2.5 Special Considerations for SPI Flash

The SPI node processes standard boot frame headers which may be concatenated so long as the flash is not being accessed in any other way during booting. This stipulation is necessary because the end of a boot frame may occur on any even bit boundary within the flash, hence 0, 2, 4 or 6 bits into an addressable byte. A straight concatenation may begin on the next bit boundary to be read. A standard BDL frame will actually generate two concatenated boot frames: One to load all nodes but the root, and the next to load the root node and begin execution. The second frame is actually generated using /ROOT but is derived from the limited BDL allowed for the root node. Initialize P to the concatenation address [705 ITS spi-exec] if the root is not going to be otherwise accessing flash and there is to be another concatenated frame.

In complex flash boots there may be several logical steps, and some of them may involve programming the root node to operate on the flash. Examples include reading the polyFORTH nucleus from flash and writing it into external SRAM, and

accessing a table of parameter values to be written at specified places in various nodes before booting the rest of their memory. In these cases the bit alignment of the end of the previous boot stream is lost and a new starting address must be written into the root node's memory so that after that step is complete the normal boot code may be started at a byte-aligned place following the end of the previous boot stream. This address, within the flash, is only known after the boot stream image has been padded, and `FORWARD` is used to patch the root node program with this address information.

To build an SPI boot stream that is unpacked for debugging purposes, use the phrase `0 S-END !` within `STREAM[ ... ]STREAM`.

6.1.2.6 *Special Considerations for Serial boot*

When preparing a 2-chip boot stream for serial delivery through node 708, it's infeasible to visit node 000 on the first chip as part of the main stream. So, if you need to program this node in any way it must be done as part of the bridge construction. The 2-chip serial pF stream generator we provide loads a ganglion into node 000. Copy the BDL for your node 000 here if you need it.

6.2 Transmitting Boot Streams

You must generate a boot stream for the particular root node you plan to present it through. While the BDL may be the same for chips to be booted through various nodes, the text of the boot streams differs because the paths necessarily differ for each root node.

The vocabulary for injecting these streams depends on the system you're running on and on the method to be used. These methods are as follows:

6.2.1 From saneFORTH

6.2.1.1 *Asynchronous Serial*

sF uses its external IDE to inject asynchronous boot streams into node 708 of the HOST chip

S-ORG S-LNG HOST LOAD !STREAM resets the chip and pumps the stream just built into it through boot node 708. Returns when all words have been transmitted. There is no flow control so completion does not prove success.

!stream is used when the serial port is already set up and the chip has already been reset. Used for second and subsequent streams when multiple streams are being injected.

!NUCLEUS transmits the polyFORTH nucleus from block zero of the pF serial disk when initializing RAM over serial port.

The asynchronous boot stream can also span both host and target. Upon completion the external IDE remains loaded and, if the stream in question left IDE code in node 708 with proper P setting, the external IDE may still be used with care and due considerations for what paths are feasible after the stream has been loaded. Use `Ta1k` instead of `TALK` to use existing serial port and avoid resetting the chip.

6.2.2 From pF/144

6.2.2.1 *Internal Streams rooted at node 207*

Streams may be injected, at very high speed, directly into the host chip after being built:

!SNORK pumps the stream just built out through the Snorkel into node 208 or 307 as indicated by the path in effect at the start of the stream. Returns when all words have been absorbed by at least the first node in the path.

Here is an example that instructs node 715 to emit an approximately 75 MHz signal on pin 715.17, assuming that nothing but ganglia have been loaded into any of the nodes along its path:

```

1581
0 ( Descriptor test)
1 ASM[ # 715 NODE ERS # 0 org
2   begin begin !b unext unext   >BIN ]ASM
3
4 0 ARRAY MYP  207 ORGN 210 TO 710 TO 715 TO -1 ,
5
6 207 STREAM[   ' MYP COURSE
7   FRAME[  715 +NODE  0 1 715 /PART
8   -1. -1. -1. -1.  -1. -1. -1. -1.  -1.  9 /RSTACK
9   x20000. x30000.  2OVER 2OVER  2OVER 2OVER  2OVER 2OVER
10                                2OVER 2OVER  10 /STACK  IO /B  0 /P
11   ]FRAME  ]STREAM
12 !SNORK

```

This example instructs node 500 of the target chip to emit a similar signal on its pin 500.17, assuming that the bridge has already been installed and activated, and that there are no obstructions along the path. Note the use of ORGN to make the hop between nodes 400 and 10400:

```

1582
0   ( 2-chip test)
1 ASM[ # 10500 NODE ERS # 0 org
2   begin begin !b unext unext   >BIN ]ASM
3
4 0 ARRAY MYP  207 ORGN 407 TO 400 TO  10400 ORGN 10500 TO  -1
5
6 207 STREAM[   ' MYP COURSE
7   FRAME[ 10500 +NODE  0 1 10500 /PART
8   -1. -1. -1. -1.  -1. -1. -1. -1.  -1.  9 /RSTACK
9   x20000. x30000.  2OVER 2OVER  2OVER 2OVER  2OVER 2OVER
10                                2OVER 2OVER  10 /STACK  IO /B  0 /P
11   ]FRAME  ]STREAM
12 !SNORK

```

For the next example, we begin with a flash boot that does NOT load anything but polyFORTH and, if desired, the bridge. This demonstrates that polyFORTH can load the ethernet cluster from within a live chip. It is a torturous procedure because we must initially do a 9 LOAD for serial clock (HOME 9 LIST and edit is the best way to go) :

```

1583
0 ( Add Ether to pF)
1 STREAMER LOAD  1 CONSTANT ?CLK  1 CONSTANT ?ETH
2
3 0 ARRAY MYP  207 ORGN 208 TO 108 TO 110 TO 10 TO 17 TO
4   617 TO 616 TO 116 TO 115 TO 415 TO 414 TO 114 TO 111 TO -
5 1 ,
6
7 207 STREAM[   ' MYP COURSE
8   FRAME[ ( Clock/Ether)  ENIC 1+ LOAD  ]FRAME  ]STREAM
9   !SNORK
10
3983
0 Assumes flash boot only has polyFORTH and, if desired, bridge.
1   This loads and activates the ethernet NIC Mk1 and the 10MHz
2   clock.  Because of the latter several steps are involved:
3

```

```

4      1.  Reset and hit space.
5      2.  HOME 9 LIST and edit to use serial clock.  HI
6      3.  AFORTH 1583 LOAD ... and link should become active.
7      4.  RELOAD hit space
8      5.  HOME 9 LIST and edit to use Ethernet clock.  HI
9      6.  ETHER LOAD
10
11 ETHER may in fact be loaded on top of AFORTH, or presumably
12 vice versa, but conflicts on double constants may exist.

```

6.2.2.2 Asynchronous Serial to Another Chip

6.2.2.3 Synchronous Serial to Another Chip

6.3 Burning Flash

By default, when generating a stream for node 705, both saneFORTH and pF/144 pack the stream into bytes that may be written to a flash memory starting at its absolute location zero. This section describes the procedures for writing that stream image into a local or remote flash.

6.3.1 Burning Flash from saneFORTH

6.3.2 Burning flash from pF/144

Code is provided for burning one's own flash (for example, the SPI flash that boots the Host chip on an Evaluation Board) or for burning the flash on another chip with which pF is communicating using external IDE.

6.3.2.1 Burning pF's Own Flash

Burning one's own flash is straightforward; all we need do is to copy the stream image to the front of the flash starting at absolute block zero (normally mapped at 40 DRIVE). After generating the stream, simply execute the following word to rewrite flash boot on your own system:

WRITE-FLASH copies the packed boot stream just generated to the boot area of the current system's flash, at absolute block 48000 and regardless of the current **OFFSET**.

Warning! This procedure assumes the flash boot area begins at absolute block 48000 in the **UNITS** mapping that is currently active on the pF/144 system. If you are using some other mapping you will need to attend to the definition named **FLASH**.

6.3.2.2 Burning Flash on Another Chip

6.3.3 Erasing Flash

Flash needs to be erased before being written. Unless otherwise noted, all of our operations that write flash take care of the necessary erasure. If you need to erase flash without writing it, study the code in **BULK** which uses **FERS** with absolute starting block number and count on the flash (remember that 0 DRIVE starts at block 128 absolute.)

The smallest unit of flash that can be erased on the Evaluation Board is 4K bytes, so erasing always starts on a 4K byte boundary and the number of bytes erased will be rounded up to the nearest 4K as well.

6.4 Auditing Streams

When working with new stream structures it's useful to compare the stream just made with a reference. Mechanism for doing this is included in STREAMER. Standard areas involved in these operations are 4740, where a packed flash stream may be saved, and 2340, where an unpacked stream of up to 60 blocks (14760 words) may be saved.

UNPACK (n) unpacks the given number of blocks of flash stream from 4740 to 2340.

STASH saves the streamer's current stream buffer to 2340, limited to 60 blocks.

?STREAM dumps the current *unpacked* stream buffer in hex, highlighting each word that differs from the reference stream in the area at 2340. In saneFORTH the reference stream is at 2340 on the serial disk used by pF/144.

?FLASH dumps the current stream buffer in hex *packed for flash*, highlighting each word that differs from the current boot flash. In saneFORTH the reference stream is at 4740 on the serial disk used by pF/144.

.LOADING available after loading block 1619. Displays what the most recently generated unpacked stream does based on the content of the STREAMER's tables.

6.4.1 Getting Streams from Flash pF/144 ONLY

The simplest way to obtain a stream made by colorForth is to burn the stream into flash from colorForth, boot to it, and then copy it to the comparison area. With standard mapping (SPI flash for 24000 blocks at absolute zero, followed by 24000 blocks of serial disk, followed by 128-block flash boot area), this can be done as follows:

1. 0 DRIVE DISKING LOAD
2. 48000 24000 4740 + nn BLOCKS (to place it on serial disk)
3. 48000 4740 nn BLOCKS (to place it on SPI flash disk).

6.4.2 Getting Streams from colorForth

When converting an existing body of colorForth based code to aF-3, it is prudent to use the colorForth stream for loading that code as a reference. The following steps accomplish this:

1. In colorForth, **compile** all of the code of interest.
2. Say **bnamed mystream** (or other name of your choice) to output file name.
3. Edit the stream generation block to abort with a bad word after the word **stream**
4. Generate stream, let it abort and note the address and count found there.
5. Say **32768 nnc + nnc + 4800 wback** to write a 4800-block file starting with the stream. Only the first few blocks are relevant, but this simplifies file mapping into the sF system.
6. Move this file into the **pf** directory
7. On the sF system, in block 149 temporarily replace **OBJ-AF3** with your file's name, **FLUSH** and **RELOAD**.
8. Use utility not written yet and conventions not made yet to capture and save the stream.
9. In block 150, restore the **OBJ-AF3** file name, **FLUSH** and **RELOAD**. If you are using the new project packaging mechanism, you will probably need to edit your **custom.txt** file instead.
10. Additional steps to be determined.

11. Remove the change you made to the stream generation block in colorForth.

7. Simulation Testing with SOFTSIM

SOFTSIM (the Software Simulator) is a program that simulates the actions of Green Arrays computers, *in two GA144 chips*, at a high level. The charter of this simulator is to achieve the same results in values but not in timing, either relative to wall time or between nodes, as would a real chip. There are coarse delays simulated for long operations such as memory read/write, but these are only provided to aid in perspective.. Tutorial follows.

7.1 The Display

After starting aF3, type **SOFTSIM LOAD** at the command line. The command prompt window will be enlarged (you may need to select a smaller font in order for this to succeed; we are not permitted to exceed the screen dimensions with that enlargement.) Here is an example of the display you should see. In the default layout shown below, there are three sections which help you "drill down" into the chip. In the upper right corner is a **full chip view** showing two GA144s as 8x18 arrays of computers, with chip 1 on top. On the left is an **overview** showing the major status and registers of a rectangular section of the chip's nodes (4x8). Between these sections is a detailed view of a **focus node**. The lower right-hand part of the display is a normal 25x80 FORTH terminal for control and programming.

The characters in the full chip view are # for nodes in the *overview*, O for the *focus node*, and = for all others. Background of nodes in this view are blue if suspended, yellow if active, and red if the node is presumed "dead" after executing an undefined operation. These colors also apply to node numbers in the overview and focus node sections.

```
arrayForth 3.0144A12
V      V      V      V      V      V      V      V      00008  p= 25 io=04291 b=r-l a=0015D io      = = = = =
302a11 303a11 304a11 305a11 306a11 307a11 308a11 309a11      pins= 0010 data=      = = = = =
4fetch 4fetch 4fetch 4fetch 4fetch 4fetch 4fetch 4fetch      >.<      i=01A8A @b ! a .      2AAAA      = = = = =
i115A5 i115A5 i115A5 i115A5 i115A5 i115A5 i115A5 i115A5      loc= 24 slot=0 @b      2AAAA      = = = = =
>0a11< >0a11< >0a11< >0a11< >0a11< >0a11< >0a11< >0a11<      SUSPENDED      2AAAA      = = = = =
a2AAAA a2AAAA a2AAAA a2AAAA a2AAAA a2AAAA a2AAAA a2AAAA      2AAAA      = = = = =
b io b io b io b io b io b io b io b io      1D 134A9 call A9      2AAAA      = = = = =
o200AA o200AA o200AA o200AA o200AA o200AA o200AA o200AA      1E 134A9 call A9      2AAAA      = = = = =
r2AAAA r2AAAA r2AAAA r2AAAA r2AAAA r2AAAA r2AAAA r2AAAA      1F 134A9 call A9      2AAAA      = = = = =
t2AAAA t2AAAA t2AAAA t2AAAA t2AAAA t2AAAA t2AAAA t2AAAA      start 20 04B12 @p b! @p .      2AAAA      = = = = =
s2AAAA s2AAAA s2AAAA s2AAAA s2AAAA s2AAAA s2AAAA s2AAAA      21 001F5 @b + + ;      r2AAAA      = = = = =
^      ^      ^      ^      ^      ^      ^      ^      22 0015D @b + ex ;      = = = = =
V      V      V      V      V      V      V      V      23 2A9B2 a! . . .      t2AAAA      = = = = =
202-l- 203-r- 204-l- 205a11 206-r- 207-u- 208a11 209a11      cmd 24 01A8A @b ! a .      s2AAAA      = # # # # #
0 @ 0 @ 0 @ 4fetch 4fetch 0 @b 4fetch 4fetch      25 2FDB2 >r @p . .      2AAAA      = # # # # #
i03B40 i03B40 i03B40 i115A5 i00000 i01D93 i115A5 i115A5      26 00007 @b and @b @p      2AAAA      = # # # # #
>0 21 >0 21 >0 21 >0a11< 0-r-< 0 38 >0a11< >0a11<      27 01B03 @b !b @b dup      2AAAA      = # # # # #
a00175 a001D5 a00175 a2AAAA a2AAAA a2AAAA a2AAAA a2AAAA      28 087C2 !b 2/ 2/ .      2AAAA      = # # # # #
b -r- b -l- b -u- b -u- b -u- b -u- b io b io      29 3EAB2 and a! . .      2AAAA      0.5796 ~us
o204AA o204AA o204AA o200AA o200AA o204AA o200AA o200AA      2A 02CAA @r> a! . .      00007      414 cyc
r2AAAA r2AAAA r2AAAA r00002 r2AAAA r2AAAA r2AAAA r2AAAA      2B 0B724 ! jump 24      2AAAA
t2AAAA t2AAAA t2AAAA t2AAAA t2AAAA t2AAAA t2AAAA t2AAAA      last 2C 134A9 call A9      2AAAA
s2AAAA s2AAAA s2AAAA s2AAAA s2AAAA s2AAAA s2AAAA s2AAAA      ^      ^      ^      ^
102-r- 103-l- 104 2E 105-l- 106-r- 107-d- 108-l- 109-r-
0 @ 0 @ 4fetch 1 !b 2 @b 2 @b 0 @b 0 @b 0 @b
i03B40 i03B40 i1B424 i05B05 i09B05 i09B02 i01A55 i01A55
0 21< 0 21< 1 2F 0 04> 0 3E< 0 1A >0 07 >0 11
a001D5 a00175 a001D5 a001D5 a2AAAA a001D5 a001D5 a00175
b -l- b -r- b io b -l- b -r- b -d- b -l- b -l- b -r-
o204AA o204AA o304AA o378AA o27CAA o200AA o240AA o250AA
r2AAAA r2AAAA r2AAAA r00032 r00175 r0001C r001D5 r00175
t2AAAA t2AAAA t20954 t011DC t00000 t20000 t2AAAA t20003
s2AAAA s2AAAA s012AB s00002 s00000 s15555 s2AAAA s2AAAA
^      ^      ^      ^      ^      ^      ^      ^
V      V      V      ^      ^      ^      ^      ^      +ph 0010 +ph
002rd1 003rd1 004rd1 005-d- 006-d- 007 40 008-r- 009-r-
4fetch 4fetch 4fetch 1 !b 1 !b 0unext 0 @b 0 @b
i115B5 i115B5 i115B5 i05B40 i05B40 i01B08A i01A8A i004DA
>0rd1< >0rd1< >0rd1< 0 03 0 03 0 41 >0 25< >0 27
a2AAAA a2AAAA a2AAAA a2AAAA a2AAAA a00141 a0015D a00141
b io b io b io b -d- b -d- b -l- b -l- b -r-
o202AA o202AA o202AA o272AA o372AA o102AA o04291 o052AA
r2AAAA r2AAAA r2AAAA r2AAAA r2AAAA r0001D r2AAAA r2AAAA
t2AAAA t2AAAA t2AAAA t2AAAA t2AAAA t3557F t2AAAA t00003
s2AAAA s2AAAA s2AAAA s2AAAA s2AAAA s00141 s2AAAA s2AAAA
^      ^      ^      ^      ^      ^      ^      ^
00515 00778
```

7.1.1 Meaning of the Data Displayed

For simplicity, the simulation assumes a fixed cycle time. All instructions begin at a cycle boundary, including resuming after suspension or completing the delay for a long operation. All parts of the display show conditions as they are before any node has begun executing its next cycle. This means the opcode shown is about to be executed if the node isn't suspended, and registers/stacks are as that next opcode will find them. While an instruction fetch is occurring, the opcode reads **fetch**. Various instructions involve time delays, in which case the opcode shown will be the one that is still executing during the delay (the delay indication differs between overview and focus node views.)

Please remember that this is not actually how the chip works! The purpose of SOFTSIM is to assist in visualizing and debugging code at a very high level, not to accurately simulate the timing of a chip made of asynchronous computers, each of which will due to process variation be running at a slightly different speed, and responding to stimuli that are synchronized only with the sources of those stimuli, not with any fixed clock. If your code respects the rules pertaining to multiport read and write operations, it should give the same results in SOFTSIM as it does in the real chip... but not in the same amount of time, and certainly not with synchronization of events between nodes that are not talking to each other or to the same internal or external central points. If on the other hand your code is pushing those rules, it could appear to work in SOFTSIM but fail in the real chip, or vice-versa. It is important to always keep this in mind when making use of SOFTSIM.

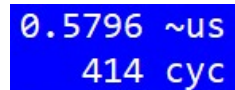
Likewise, I/O testbeds, discussed later, must be written and used with the understanding that the simulation time scale is intentionally and practically not reliable. Attempts to stress the chip with external inputs whose speeds approach the chip's limits will easily produce misleading results. Using the simulator to test logic is fine; explore timing limits with real chips and apply margins to account for variation in chip performance due to process, voltage, temperature and aging.

Where addresses are shown in these displays, they are formatted either as two digit hex numbers for RAM or ROM; or a three character string for I/O addresses. In the latter case **all** means **rdlu**. Other forms are self-explanatory.

The display shown below represents "cycle" 505 of chip 0 loaded with polyFORTH and the Ethernet NIC, set to use a 10 MHz crystal, but with no I/O testbeds (so external SRAM is not connected and what's being read from it is garbage). The details of the sections, and the vocabulary for arranging the display as you like, are discussed in the following sections.

7.1.2 "Time" and Cycle Count

Under the full chip views are two numbers. The first, labeled **~us**, is an extremely rough estimate of the elapsed time since simulated RESET. The second, labeled **cyc**, is the number of simulation cycles that have run since RESET. Because, as noted above, "cycles" do not exist in the real chip either within a node or between nodes, the number of cycles is an abstraction. The approximate elapsed time is accumulated at 1.4 ns per "cycle", so it too is an abstraction. That said, the time is likely within an order of magnitude of reality so it can be useful.



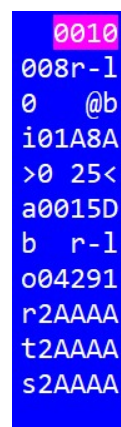
7.1.3 The Overview Section

This section shows a high level summary of what's going on in each of 32 nodes in an 8x4 array. The array may be moved around on the chips; the nodes currently displayed are shown on the full chip view as the character **#**. As an example let's look at node 008 in the example above.

The lines, starting from 1 on top, indicate: 1) that we are driving pins 17,5,3,1 as shown; 3 high means SRAM read. 2) This is node 008, not running, and the "address bus" is r-l-. 3) Next op is @b in slot 0. 4,6,8,9,10,11) Registers I (instruction), a, io, r, t, s. 5) no delay, register P is x25 (op came from x24). 7) b register, interpreted symbolically as an address, is r-l-. The reason the node isn't running is that it is suspended, waiting on a fetch from RIGHT or LEFT.

Refer to the table in the next section for the complete legend for nodes in this view.

I/O pins or ports are shown on the top and/or bottom of the overview. GPIO and other normal pins are depicted as 0 or 1 for low or high with color attribute indicating pin mode (white on blue high impedance input; white on grey input with weak pull-down; white on magenta output.)



Each node's depiction shows a list of the values of registers and opcode names representing the current state of that node. The state depicted is *before* an instruction is executed; the opcode shown will be executed next, and all the registers are shown as they will be when the opcode begins execution. When the F18 is suspended waiting for port or pin wake-up, the operation shown is the one that's suspended. For certain fetch or store operations there is a multi cycle delay, in which case the operation shown is the one whose execution is not yet complete. If an instruction word is being fetched, the current opcode will read **fetch** . Each node shows twelve lines. From top to bottom, using node 007 from the above display as an example, they are:

Row	Example	Name	Description
1	<i>C pppp</i>	Port r/w and pins.	<i>C</i> indicates reading (V) or writing (^) the port leading to the "North". <i>pppp</i> is blank unless the node has pins. +ph and -ph indicate phantom pin high or low. Pin states are shown with pin 17 on left and pin 1 on right. Each pin is shown with its voltage as high (1) or low (0). The mode of the pin is shown by color and background mapping. Output is white on magenta. High impedance input is white on blue. Input with weak pull-down is white on grey.
2	<i>yxxbus</i>	NODE/Abus	<i>yxx</i> is node number within either chip. As noted above, it will be black on yellow if active, white on blue if suspended, or white on red if locked up. The "address <i>bus</i> " value shown (symbolically if in the I/O range) is that used for the most recent (or ongoing) memory operation.
3	<i>sOpcod</i>	Slot/Opcode	The slot selector and next/current opcode to be executed in the next cycle. The opcode is fetch if we are doing an instruction fetch.
4	<i>i03B0A</i>	I	The instruction register
5	<i>WdpppE></i>	M/P	<i>W</i> and <i>E</i> indicate writing or reading the "West" and "East" ports using <> to show read/write direction similarly to use of ^ and V for North/South. <i>d</i> is the delay counter. If it's >1, the operation shown will not complete this cycle. <i>ppp</i> is the P register (program counter) value, defining the next address to be fetched/stored by @p !p or instruction fetch.
6	<i>a00115</i>	A	18 bit pointer register A.
7	<i>b -1-</i>	B	10 bit pointer register B, symbolic if known.
8	<i>o102AA</i>	IO	The write-only part of the IO register.
9	<i>r2AAAA</i>	R	Top of return stack.
10	<i>t00141</i>	T	Top of data stack. If carry latch is set, "*" replaces "t".
11	<i>s0015D</i>	S	Second on data stack.
12	<i>Cdddd</i>	Pins and/or ports	<i>C</i> indicates reading (^) or writing (V) the port leading to the "South". <i>dddd</i> if nonblank is the value of the data register/bus/pins. When white on blue the port or device is in input mode. When white on magenta, the port or device is in output mode.

7.1.4 The Focus Node Section

Between the overview and the full chip view lies a detailed depiction, expanding on the overview, of the current **focus node**. Field depiction and color coding are the same except as indicated. Here is node 8 at the moment of the above example.

The full **cyxxx** node number is shown, highlighted for activity or lock-up. Registers p, io, b and a are shown on the top line; p and b are shown with symbolic addresses when relevant. If a is in the I/O range, its full hex value is also shown symbolically.

Pins and data in the next line are the same as those in the top and bottom rows of the overview.

The next line shows the I register with its disassembly.

The address, slot and opcode shown reflect the instruction to be executed in the next cycle, or the previous instruction being continued due to suspension (for I/O) or delay (for operations like memory). Note that in normal operation P advances after each fetch or store in memory using P, so unless the node is executing from a port, "p" and "loc" will normally differ. Beneath that line may appear **SUSPENDED**, **DELAYING** or **LOCKUP!**.

To the left of these lines is a dot. If there are any port communications going on, the symbols used for the same purpose in the overview appear on the cardinal sides of the dot. "Arrows" pointing to the dot are reads, away writes.

Beneath these lines is a dump of 16 contiguous cells of memory showing labels if any, address, value of the data and a disassembly of that value interpreted as an instruction word. If the address in P is visible, it is highlighted black on yellow.

The right side of this view shows the return stack on top, growing downward, and data stack below, growing upward. Carry is shown in the same way as in overview: Label for register t changes to * when carry set.

Operator functions are provided to select a focus node, and to toggle between the current and most recent focus nodes. For any given node, by default the memory dump tracks P to ensure the next cell of the instruction stream is visible. Operator functions exist to override this by specifying the starting address which is then frozen, and to release this freezing of the dump origin.

7.2 Loading Code to Simulate

When SOFTSIM is loaded, node RAMs are only initialized as they typically appear on power-up (x15555) and the nodes are all set to their P values as of RESET (the simulator presently uses xA9, warm, for non-boot nodes, for simplicity). If you allow this to run, all nodes will, in not many ns, be **SUSPENDED** (asleep) as is the case with the real chip.

At present, there is only one fully supported method for initializing the chip with code to run. SOFTSIM directly understands and processes Boot Descriptor Language (see 6.1.1). So, after loading SOFTSIM, load some BDL. For example, see block 1674, which loads polyFORTH virtual machine, SRAM cluster, and the Ethernet NIC. This body of code will not do much without I/O testbeds but it is a place to start. See below for another example.

```

00008  p= 25 io=04291 b=r-l a=0015D io
      pins= 0010 data=
>.<   i=01A8A @b ! a . 2AAAA
      loc= 24 slot=0 @b 2AAAA
      SUSPENDED 2AAAA
      2AAAA
      1D 134A9 call A9 2AAAA
      1E 134A9 call A9 2AAAA
      1F 134A9 call A9 2AAAA
start 20 04B12 @p b! @p . 2AAAA
      21 001F5 @b + + ; r2AAAA
      22 0015D @b + ex ; 
      23 2A9B2 a! . . . t2AAAA
cmd    24 01A8A @b ! a . s2AAAA
      25 2FDB2 >r @p . . 2AAAA
      26 00007 @b and @b @p 2AAAA
      27 01B03 @b !b @b dup 2AAAA
      28 087C2 !b 2/ 2/ . 2AAAA
      29 3EAB2 and a! . . 2AAAA
      2A 02CAA @ r> a! . 00007
      2B 0B724 ! jump 24 2AAAA
last   2C 134A9 call A9 2AAAA

```

7.3 Operating the Simulator

At any time, you may type **HELP** to display a help screen with reminders of these operating procedures. The vocabulary for operating is as follows:

OVIEW enables the IJKL keys to move the 8x4-node overview rectangle around. You will see feedback in both the full chip view and the overview area. Any key other than IJKL exits this dialog.

SEE (nn) selects a new Focus Node (cyyxx) and remembers the previous one.

OTHER toggles between the current focus node and the one most recently focused.

MEM (a) sets new starting address for memory display of current focus node and freezes it.

-MEM un-freezes memory display of current focus node so it will again track P.

Z takes a single step of all nodes and updates entire display.

FAST (n) runs n steps updating only the display of "time" and cycles. Full display update when done.

SUPER (n) like FAST but does not update the display at all until done.

SS Sets for operator-initiated single steps by striking any key except ENTER, which punches out.

BREAK (a s nn) sets breakpoint for node nn when next op to execute is address a slot s.

-BREAK (nn) clears any breakpoint for node nn.

7.3.1 About Breakpoints

Each node may have one and only one breakpoint defined. When any node is about to execute the instruction from the address and slot given, SOFTSIM will stop before starting to execute the entire cycle in which that instruction would begin (or continue) execution and display that node as the focus node.

If you specify an instruction that may suspend, it will continue to stop simulation before its execution has started and will not re-breakpoint during suspension. The same is true of operations such as memory fetch/store that can be delayed. Some thought is justified in choosing breakpoints.

7.3.2 Initialization

Block 1698 is an example of loading SOFTSIM, loading chip 0 with polyFORTH and the Ethernet NIC, using the TB-SRAM testbed that simulates a 2Mb external SRAM that's initialized with the polyFORTH nucleus. It will run up to the point of waiting for an autobaud space key in node 200.

Another example is block 1737 which loads SOFTSIM, assembles a simple program, loads it into node 000, and sets up the view for simulation.

7.4 Testbeds

Testbeds in SOFTSIM are pieces of code that simulate external circuitry well enough to exercise the F18 code. Several are provided and by default compiled with SOFTSIM. Each testbed provides a vocabulary to instantiate it for one or more interfaces. For example, **TB-SRAM** instantiates a 2MB external SRAM interfaced with the 40 pins of nodes 7, 8 and 9.

You may write your own testbeds for use with SOFTSIM. The linkage is via **nTB** for each node. This vector is @EXECUTEd after each instruction cycle has been executed on a node but before the end-cycle updating, suspension or delay processing have been done.

7.5 Interactive Testing with Softsim

You are invited to study the code of SOFTSIM and to make use of that information if you wish to probe details of a node's state that are not displayed in our views. The vocabulary for doing this is not documented presently because SOFTSIM in arrayForth 3 is still under development and thus is subject to change.

8. Reference Material

8.1 F18A Code Library

GreenArrays has written, tested and used a considerable Library of generally useful F18A code; this varies from single utility nodes, such as the Perfect Wire, to multi-node clusters such as the Ethernet NIC. Some of the code is useful anywhere on the chip, while other code such as, for example, the SRAM cluster, depends on the I/O and geometric properties of the nodes for which it is designed. Every bit of this code is distributed as source. Where practical, such code is pre-assembled and is distributed with the system as object code in virtual bins (1600 to 3200) so that it may be used readily by tools such as the IDE or included in applications via boot descriptors. In other cases, such as the Ethernet Cluster or the polyFORTH Virtual Machine, enough nodes are involved that it makes more sense to assemble it for the target nodes when needed by an application being built.

This section documents the code, and its usage, for each such element in the Library.

8.1.1 Perfect Wire (any node)

It's often necessary to use a node as a simple wire to connect ports that are not physically adjacent. In the simplest case, when the use is permanent (until chip reset) and the data move in only one direction, there is a demonstrably perfect solution: A single instruction word program that, once fetched from memory, runs entirely in the Instruction Register **I** and is an infinite loop with minimal latency and minimal jitter (one word of every 262144 transferred takes an extra unext time.)

The instruction word may be written as `A[ begin @ !b unext unext ]`. This single word is provided as location zero of Virtual Bin 1802 and may be placed in an application by the following descriptor macro:

+wire (s d nn) Places perfect wire in RAM location zero of the given node. Initializes **a** and **b** to the source and destination addresses **s** and **d**. Initializes the return stack with nine -1s.

8.1.2 64-word Delay Line (any node)

A node may be programmed to serve as a 64-word delay line with throughput on the order of 8 memory cycles (~40 ns in theory, 52 ns in practice) per data word, and this performance remains the same no matter how many such nodes are placed in series (although latency from input to output when a word is pushed into a quiescent line does depend on the number of nodes).

Each data word is pushed into the first delay line node surrounded by two instruction words, and each data word between nodes and emitted at the far end of a delay line is also surrounded by the same two instruction words. No Virtual Bins are used because no code in RAM or ROM is involved at all. Delay line nodes are only committed to this use in the sense that they require special initialization of the data stack (with instructions) and registers **a** (as any valid RAM address), **P** (as the source port) and **B** (as the destination port). So, at any time, a delay line node may be recommissioned via port execution (by default, we set it up with the source port address in **P**, but any multiport address may be used if desired.)

To place a delay line node in an application, use one of the following descriptor macros:

+dly (s d nn) Initializes **P** and **B** of the given node to the source and destination addresses **s** and **d**. Sets **A** to zero. Initializes the data stack with 5 copies of this instruction pair:

`A[ !b @p @ ]` in **T** and `A[ !b !+ !b ]` in **S**.

+dx (nn n) Places **n** delay line nodes starting with node **nn** and moving horizontally (to the East if **n** positive, to the West if **n** negative). Source and destination ports of all nodes including the first and last proceed in the direction indicated.

+dy (nn n) Places **n** delay line nodes starting with node **nn** and moving vertically (to the North if **n** positive, to the South if **n** negative). Source and destination ports of all nodes including the first and last proceed in the direction indicated.

The following code shows one way to insert a datum into the delay line:

```
: shove ( n)    <port> b!  @p !b A[ !b @p @ ] ] ,  
    !b @p !b ; A[ !b !+ !b ] ] ,
```

At the output end, here's one way to receive a datum:

```
: suck ( - n)    @ drop @ @ drop ;
```

As you can understand by studying the above, this is an excellent example of the remarkable capabilities of the F18A computer. The "program" for a delay line node is the pair of instructions pushed into its source port and is executed in that port; these two instructions surround the incoming datum. What this program does is to send, from its own data stack, exactly the same two instructions to the next node; however the datum between those two instructions is the oldest word in this node's RAM, which is replaced by the incoming data word.

8.1.3 Synchronous Port Bridge (300 and others)

An almost-transparent port bridge in bin 1904 uses nodes 300 on two chips to simulate a COM port between the UP ports of nodes 400 on each chip. The exception about transparency is that a node 400 cannot tell by inspecting its IO register whether the other chip's node 400 is actually reading its UP port. This bridge code is used by External and Internal IDE, by boot streams, and afterward by applications in 2-chip environments. For discussion, see 5.1.6 above.

8.1.4 Ganglia Mark 1 (any node)

8.1.5 Ganglia Mark 2 (any node)

By default we like to fill all nodes with this versatile messaging fabric, which is capable of exchanging up to 262k word payloads and replies between any two nodes on one or more chips that can be reached via any path through contiguous nodes running the ganglia and connected by COM ports or equivalent, such as the synchronous port bridge or any other bridging mechanism that supports flow control. These Ganglia are documented fully in App Note AN017, *Ganglia Mark 2*.

Object code is stored in Virtual Bins 1714, 1715, 1716 and 1717, one for each of the four physical orientations of F18A nodes. To fill both chips with ganglia, use the descriptor macro **GANGLIA** as *an early step* (before any memory load descriptors are processed) in a new frame definition.

8.1.6 SRAM Mark 1 Cluster (7, 8, 9, 107, more)

Code for providing services to, and coordination among, up to three Masters for a 1 MWord external SRAM. See AN003, *SRAM Control Cluster Mark 1*, for detailed documentation of usage and of internals.

8.1.7 Snorkel Mark 1 (any SRAM client node)

Code for a programmable DMA channel that may act as one of the Masters for the SRAM cluster. See AN010, *The Snorkel Mark 1*, for complete details.

8.1.8 IDE Components (708, 300 + any nodes)

The External IDE loads these components into the nodes it uses to reach into a chip and touch any nodes to which a path of available nodes exists.

8.1.8.1 IDE Asynchronous Interface (708)

Code for node 708 to provide services for external IDE. See AN019, *Interactive Development Environment*, for discussion of internals.

8.1.8.2 IDE Synch Interface (300)

Code for node 300 to provide services for external IDE. See AN019, *Interactive Development Environment*, for discussion of internals.

8.1.8.3 IDE Special Wire (any node)

Code for the nodes, if any, between the root node and the end node of an external IDE path. See AN019, *Interactive Development Environment*, for discussion of internals.

8.1.8.4 IDE End Node (any node)

Code for the final node, if any, between the root and target nodes of an external IDE path. See AN019, *Interactive Development Environment*, for discussion of internals.

8.1.9 Boot Stream Components

Boot stream generation is done entirely by distributed utilities. The components should be changed only if absolutely necessary and then only with great care.

8.1.9.1 SPI Flash Speed-up (705)

The ROM code for SPI flash booting uses, initially, a very slow timing parameter so that it can interoperate with slow devices. The first boot frame included in a boot stream for use from flash should begin by dropping a short program into node 705 and executing it to adjust this parameter for a higher speed. This program is provided in Virtual Bin 1608 for use by GreenArrays utilities.

8.1.9.2 Synch Boot Master

This code for node 300, in bin 1901, can transmit boot frames to a GreenArrays synchronous boot node. It's used as a step in the set-up of a Port Bridge and for other purposes.

8.1.9.3 Port Bridge Set-up

This code for node 400, in bin 1907, is used ephemerally along with the Synch Boot Master in node 300 and a copy of the code for the Synchronous Port Bridge in node 500. Once all three of these nodes have been loaded it initializes node 300's I/O, resets the other chip via node 500 on pin 500.17, loads the port bridge code into node 300 of the other chip and starts it up, then loads the same code into our chip's node 300 and starts it as well. After this is completed nodes 400 and 500 may be re-loaded with application code, filled with Ganglia, or whatever else you desire.

8.1.9.4 SRAM Initialization (specific nodes)

8.1.10 polyFORTH Cluster (specific nodes)

This body of F18 code implements a virtual machine that runs a pseudo-instruction set from external SRAM. It includes a 2-node basic VM with four nodes of extension coprocessors, serial I/O through nodes 100 and 200 with associated wiring, and SPI flash mass storage. Dependencies include the SRAM Control Cluster Mk1, Snorkel Mk1 and Ganglia Mk2. Support is provided for the Ethernet NIC and for the various 10MHz frequency references it supports. For details about this body of code, please Chapter 10, *polyFORTH*.

8.1.11 Ethernet NIC Cluster (specific nodes)

This body of F18 code, documented in AN007, implements a 10baseT Full Duplex autonomous NIC that operates on data structures in external SRAM as a virtual DMA device. The object code is temporarily stored in bins corresponding with the nodes in which the code resides when loaded. In due time it will be moved to library bins.

8.1.12 ATS Mark 1 (Serial IDE based)

The Automated Testing System is documented in DB007. The bulk of its F18 test code, in the form of "creeper" packages, is stored in bins 2000..2208, although a couple of components are in bin "row" 1900.

8.1.13 ATS Mark 2 (On-chip based)

Words.

8.1.14 eForth (specific nodes)

Words.

8.1.15 Documented Examples

Words.

8.1.15.1 Practical Example used above

Words.

8.1.16 Additional Test Code

Words.

8.2 Bin Assignments

"Bins" are receptacles for object code and are numbered like nodes in the chips. Bins 000 through 717 contain code that is by default destined for the nodes on the host chip; future releases will vacate this range completely. Bins 800 through 1517 map onto the target chip. Bins 1600 through 3117 are reserved for utility and tool code, and are managed by GreenArrays. Their purpose is to allow this code to be compiled and available for use at any time without interfering with application code. This table documents the assignments in effect as of the release identified above; yellow highlight indicates conversion to and validation of af-3 source.

1600 IDE async root node	1900 ats/ide analog	2200 ATS + (Mark)
1601 IDE sync root node	1901 ats/ide sync boot master	2201 ATS Ret&Data Stacks v2 (Mark)
1602 IDE wire node	1902 ats/ide test frame	2202 ATS Data stack v2 (Mark)
1603 IDE end node	1903 ats/ide test frame	2203 ATS + v2 (Mark)
1604 ide all nodes template	1904 Sync Bridge (with flow)	2204 ATS and (Mark)
1605 Snorkel Mk1	1905 ats/ide uut bridge debug	2205 ATS and v2 (Mark)
1606 SST25WFxxx Flash node 705 pF	1906 ats/ide tester bridge debug	2206 ATS xor (Mark)
1607 New flash helper 706	1907 framer bridge builder	2207 ATS xor v2 (Mark)
1608 spi speedup function	1908 Sync Bridge (no flow)	2208 ATS inv (Mark)
1609 18-bit flash r/w for 705	1909	2209
1610 18-bit flash helper for 706	1910	2210
1611 8-bit flash r/w	1911	2211
1612 Flash ops for new devices	1912	2212
1613 flash erase	1913	2213
1614 SRAM Node 107 interface (Mk1)	1914	2214
1615 - Node 007 SRAM Data Mk1	1915	2215
1616 - Node 008 SRAM Ctls Mk1	1916 ATS empty creeper	2216
1617 - Node 009 SRAM Adr Mk1	1917	2217
1700 polyFORTH Stack (106)	2000 ATS Port test	2300 <reserved creeper tests 4>
1701 - Stack down (006)	2001 ATS Port test with 2*	2301
1702 - Stack up (206)	2002 ATS RAM extensive test (Jeff)	2302
1703 Bitsy (105)	2003 ATS T&S register test	2303
1704 - Bitsy down (005)	2004 ATS T&R register test	2304
1705 - Bitsy up (205)	2005 ATS Data stack registers	2305
1706 Serial tx (100)	2006 ATS Return stack registers	2306
1707 - Rx (200)	2007 ATS LoVD RAM test (Mark)	2307
1708 - Interface (104)	2008 ATS Stacks 1 (Mark)	2308
1709 - Wire (102 and others)	2009 ATS Multiply Step & shift	2309
1710 Flash to sram for 705	2010 ATS T,S,A&R Data Path Tests	2310
1711 - Wire for 605 etc	2011 ATS GPIO pin test (no EVB)	2311
1712 - <unused>	2012 ATS Data stack (Mark)	2312
1713 - Temp SRAM code for 108	2013 ATS I/O reg latches	2313
1714 Ganglion nodes eee (Mk2)	2014 ATS Carry set/clear	2314
1715 - nodes eoo	2015 ATS @p test (Mark)	2315
1716 - nodes oee	2016 ATS R,I to Address data paths	2316
1717 - nodes ooo	2017 ATS ROM Checksums	2317
1800 <reserved for eForth/pF>	2100 ATS Parallel Port Pins	
1801 Ethernet DMA	2101 ATS @b !b (Mark)	
1802 Small tools (wire, etc.)	2102 ATS T-B (Mark)	
1803	2103 ATS Port-I (Mark)	
1804	2104 ATS Port-I Shorts 1 (Mark)	
1805	2105 ATS Port-I Shorts 2 (Mark)	
1806	2106 ATS Port-I Shorts 3 (Mark)	
1807	2107 ATS Port-I Shorts 4 (Mark)	
1808	2108 ATS Port-I Shorts 5 (Mark)	
1809	2109 ATS Port-I Shorts 6 (Mark)	
1810	2110 ATS P-R-P Call (Mark)	
1811	2111 ATS P-R-P Execute (Mark)	
1812 Flash to SRAM 8-bit	2112 ATS SERDES 001 Slave	
1813 Node 600 code (temporary)	2113 ATS SERDES 701 Master	
1814 Flash to sram for 705	2114 ATS Analog Basic checks	
1815 - Wire for 605 etc	2115 ATS 2* (Mark)	
1816 - SRAM interface for 208	2116 ATS 2/ (Mark)	
1817 - Temp SRAM code for 108	2117 ATS 2*.b (Mark)	

2400	2700	3000	GPS Correlator Mk1 Carr 1st
2401	2701	3001	" " Carrier 2nd
2402	2702	3002	" " Carrier Turnaround-1
2403	2703	3003	" " Carrier Turnaround 2
2404	2704	3004	" " Code
2405	2705	3005	" " Carrier special first
2406	2706	3006	" " Carrier special last
2407	2707	3007	GPS Strm delay fill
2408	2708	3008	GPS Strm delay empty
2409	2709	3009	GPS Osc Monitor (500)
2410	2710	3010	GPS Osc Agent (600)
2411	2711	3011	
2412	2712	3012	GPS TEMP input mux strm/data
2413	2713	3013	GPS TEMP Smart answer buffer
2414	2714	3014	GPS TEMP snork/gang sig feed
2415	2715	3015	GPS TEMP Sig gen for meas
2416	2716	3016	GPS TEMP Haleakala sF inject
2417	2717	3017	GPS TEMP absorber
2500	2800	3100	SRAM test helper node 108
2501	2801	3101	GPS Sig Input 708
2502	2802	3102	GPS FIFO Mk3
2503	2803	3103	GPS Instrumentation 705->706
2504	2804	3104	GPS Mem Mk3 608
2505	2805	3105	GPS TEMP Sig Input 709
2506	2806	3106	
2507	2807	3107	
2508	2808	3108	
2509	2809	3109	
2510	2810	3110	
2511	2811	3111	
2512	2812	3112	
2513	2813	3113	
2514	2814	3114	
2515	2815	3115	
2516	2816	3116	
2517	2817	3117	
2600	2900		
2601	2901		
2602	2902		
2603	2903		
2604	2904		
2605	2905		
2606	2906		
2607	2907		
2608	2908		
2609	2909		
2610	2910		
2611	2911		
2612	2912		
2613	2913		
2614	2914		
2615	2915		
2616	2916		
2617	2917		

000	TEMP	300	TEMP	600	TEMP
001	TEMP	301	TEMP	601	TEMP
002	TEMP	302		602	TEMP
003		303		603	TEMP
004		304	TEMP	604	TEMP
005	TEMP	305		605	TEMP
006	TEMP	306		606	
007	TEMP	307		607	
008	TEMP	308		608	
009	TEMP	309		609	
010	ETH Rx Control	310		610	
011	ETH Rx Byte swap	311		611	
012	ETH Rx Pack	312		612	
013	ETH Rx CRC	313		613	
014	ETH Rx Frame	314	ETH Tx Framing	614	
015	ETH Rx Parse	315	ETH Tx Mux	615	
016	ETH Rx Timing	316	ETH Wire **	616	ETH/CLK Osc Agent
017	ETH Rx Jitter buffer & Clean	317	ETH Tx Pin	617	ETH/CLK Osc Monitor
100	TEMP	400	TEMP	700	TEMP
101	TEMP	401	TEMP	701	TEMP
102	TEMP	402		702	
103	TEMP	403		703	
104	TEMP	404	TEMP	704	TEMP
105	TEMP	405		705	TEMP
106	TEMP	406		706	TEMP
107	TEMP	407		707	
108	TEMP	408		708	TEMP
109	ETH Slave of DMA Nexus	409		709	
110	ETH Master of DMA nexus	410		710	
111	ETH Tx Control	411		711	
112	ETH Wire **	412		712	
113	ETH Tx Unpack	413		713	
114	ETH Tx CRC	414		714	
115	ETH Link States	415	TEMP XTAL Start control	715	
116	ETH Wire **	416	TEMP XTAL Sequencer	716	
117	ETH Rx Pin	417	ETH Passive TX Oscillator	717	TEMP
200	TEMP	500	TEMP		
201	TEMP	501	TEMP		
202		502			
203		503			
204	TEMP	504	TEMP		
205	TEMP	505			
206	TEMP	506			
207	TEMP	507			
208	TEMP	508			
209		509			
210		510			
211		511			
212		512			
213		513			
214	ETH Tx Delay	514			
215	ETH Negotiation	515			
216		516			
217	ETH Rx Pull-down	517	TEMP Xtal Osc		

8.3 Examples to remember

A simple stream:

```
1581
0 ( Descriptor test)
1 ASM[ # 715 NODE ERS # 0 org
2   begin begin !b unext unext   >BIN ]ASM
3 0 ARRAY MYP  207 ORGN 210 TO 710 TO 715 TO -1 ,
4 207 STREAM[   ' MYP COURSE
5   FRAME[   715 +NODE  0 1 715 /PART
6   -1. -1. -1. -1.  -1. -1. -1. -1.  -1.  9 /RSTACK
7   x20001. x30002. x20003. x30004. x20005. x30006.
8   x20007. x30008. x20009. x30010. 10 /STACK  IO /B  0 /P
9   714 +NODE 111. 222. 333. 444. 4 /STACK
10  ]FRAME  ]STREAM
11 !SNORK
```

```
.STREAM 512
 0 10175 4DAF 121D5 78 2FAB2 5A72 4DAF 12175 208,9
 8 72 2FAB2 5A72 4DAF 12115 6C 2FAB2 5A72 210
10 4DAF 12145 66 2FAB2 5A72 4DAF 12115 60 310,410
18 2FAB2 5A72 4DAF 12145 5A 2FAB2 5A72 4DAF 510,610
20 12115 54 2FAB2 5A72 4DAF 121D5 4E 2FAB2 710
28 5A72 4DAF 12175 48 2FAB2 5A72 4DAF 121D5 711,12
30 42 2FAB2 5A72 4DAF 12175 3C 2FAB2 5A72 713
38 4DAF 121D5 2E 2FAB2 5A72 4A12 0 0 714,15
40 2E9B2 5872 9175 4BB2 15D 48B2 3FFFF 48B2 Regs
48 3FFFF 48B2 3FFFF 48B2 3FFFF 48B2 3FFFF 48B2
50 3FFFF 48B2 3FFFF 48B2 3FFFF 48B2 3FFFF 49B2
58 20001 49B2 30002 49B2 20003 49B2 30004 49B2
60 20005 49B2 30006 49B2 20007 49B2 30008 49B2
68 20009 49B2 30010 101B5 49B2 6F 49B2 DE 714
70 49B2 14D 49B2 1BC 101B5 101B5 101B5 101B5 713-
11
78 101B5 101A5 101A5 101A5 101A5 101A5 101A5 101A5 710-
208
```

9. Maintenance

This section covers procedures for maintaining the sF and pF/144 systems.

9.1 saneFORTH Maintenance

9.1.1 Bootstrapping

sF is normally started on a Win32 platform via one of three Windows executables, named in the following forms:

- **sF<vers>-aF3.exe** the normal production nucleus, with 16 MB of system memory. This executable must lie in a path whose name contains no embedded spaces because it parses its command line with simple code. Upon boot, this autoloading nucleus loads block 534 which loads block 9, defines code for accessing the command line, parses it, and INTERPRETs any FORTH code following the first space (which should be the one following the name of the executable). This program is normally started using a supplied shortcut by the name arrayForth 3 G144A12 whose command line has AFORTH appended.
- **sF<vers>-glow.exe** the normal production nucleus, but with 1.6 GB of system memory to support the GLOW design tools (which are redacted from publicly released systems.) Normally started using a supplied shortcut by the name arrayForth 3 with GLOW. As distributed the shortcut has no interpretive code.
- **sF<vers>-panic.exe** is a standard saneFORTH nucleus which does not autoloading; you will need to type HI to load block 9. It has its own shortcut as well, named arrayFORTH 3 PANIC. It is provided to give a clean saneFORTH environment that may be used, for example, to fix mistakes that have been made in either the 9 LOAD or in the AFORTH code.

9.1.2 Nucleus Maintenance

Because this is a hosted system, the nucleus that is run for saneFORTH is necessarily packaged into the Windows executable. The sF nucleus source, as well as the Target Compiler and other utilities employed to test nuclei and to generate Windows executables, are proprietary to FORTH, Inc. and/or ATHENA Programming, Inc. and GreenArrays has not been licensed to distribute them. Therefore these sections are redacted and users of arrayForth 3 are unable to maintain the underlying x86 nucleus. Special versions, if necessary, must be requested from GreenArrays.

9.2 polyFORTH/144 Maintenance

pF/144 is fully equipped to maintain and alter both its Virtual Machine and its polyFORTH nucleus. These procedures are fully documented in *DB006, G144A12 polyFORTH Supplement*. The shadows for pF/144 nucleus source, in the index page at 60, constitute its primary documentation.